放手时刻：智能体自主驱动的影响力置换排序优化

Sortify Team

推荐系统排序本质上是一个影响力分配问题：排序公式在自然相关性、广告出价、价格竞争力等竞争性因子之间分配排序影响力，而业务结果取决于能否找到各因子间最优的“兑换汇率”。然而，离线代理指标对影响力再分配的在线效果存在系统性误判——离线代理指标的大幅提升往往仅有少部分转化为在线收益，迁移率远低于预期；更为复杂的是，不同指标间的偏差模式呈非对称分布——部分指标呈乐观偏差（离线高估在线），另一些则呈悲观偏差（离线低估在线），单一校准因子无法同时修正两者。传统方案在三个层面加剧了这一困境：人工校准无法跟上数据分布漂移的速度；诊断信号相互纠缠，导致无法区分映射偏差与约束失调；且每轮优化均从零开始，历史经验无法沉淀。

本文提出 Sortify——一个由大语言模型（LLM）驱动的全自动排序优化智能体。该智能体将排序优化重构为持续的影响力置换过程，在各排序因子间自主调控影响力分配，无需人工干预即可完成从诊断、决策到参数落地的全链路闭环，并通过三项核心机制应对上述结构性问题。其一，基于 Savage 主观期望效用理论（SEU）的公理化立场——任何理性决策都需要且仅需要两个独立输入：对世界状态的信念（概率）与对结果的偏好（效用）——智能体将离线-在线迁移校准（Belief 通道）与约束惩罚调整（Preference 通道）解耦为正交维度，从架构层面切断认知偏差与价值失调的诊断纠缠，实现独立修正。其二，LLM 元控制器作为二阶理性的观察者，在框架级参数上操作——调整迁移函数截距和惩罚乘数，而非底层搜索参数——基于 20 轮（episode）的历史证据，对常规 LMS 校准留下的残差进行选择性修正。其三，智能体引入了包含 7 张关系表的持久化记忆数据库（Memory DB）以实现跨轮学习积累，为热启动和跨轮校准延续提供状态基础。

该智能体的核心评估指标 Influence Share 是一种各因子贡献之和恒等于 100% 的可分解排序影响力度量。它弥补了 Kendall Tau 等传统排名相关性指标无法归因到具体因子、无法支持因子间定量兑换的根本局限。Sortify 已成功部署于覆盖东南亚两个市场的大规模推荐平台（下文分别称为 A 国和 B 国），实现端到端全自动运行——无需人工介入。其内层优化循环以 4 小时为周期自主执行，每轮执行 5,000 次 Optuna 搜索试验。在 A 国热启动实验中，智能体在 7 轮内将 GMV 从 −3.6% 推升至 +9.2%，订单量峰值达到 +12.5%，同时 LLM 校准工作量从每轮 5 项修正收敛至 2 项。在 B 国，冷启动部署进一步验证了其跨市场泛化能力和端到端部署能力：经过两阶段搜参（共 23 轮），智能体识别出一组参数结构，在 7 天的 A/B 验证中实现 GMV/UU +4.15%、Ads Revenue +3.58%，并据此推全至生产环境。整体而言，保留于正文的证据表明该系统既能在既有校准状态上持续改进，也能在新市场中产出可部署策略。

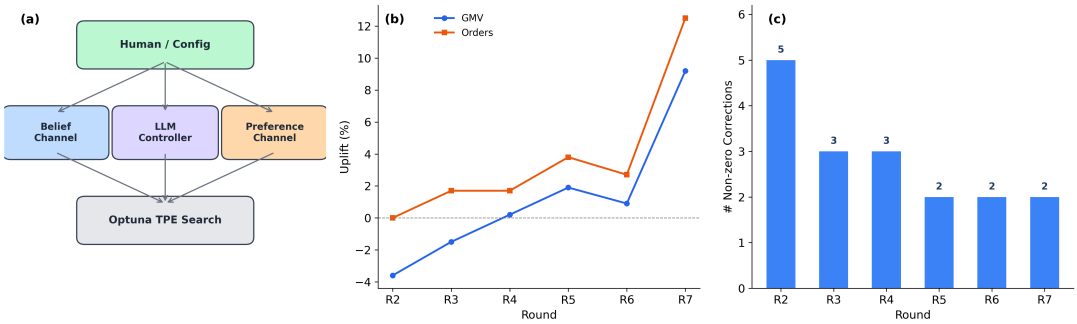

图 1 | 三面板概览——(a) 双通道架构示意图，(b) 在线 GMV/Orders 提升随轮次变化趋势，(c) LLM 修正收敛曲线

目录

1 引言 **5**

2 系统架构 **8**

2.1 总述 8

2.2 Influence Share 与参数搜索引擎 9

2.2.1 动机：超越 Kendall Tau 10

2.2.2 Influence Share 定义：从对称群到影响力置换算法 10

2.2.3 多目标搜索建模 12

2.2.4 搜索引擎 12

2.3 Belief 通道：离线-在线迁移校准 13

2.3.1 动机 13

2.3.2 线性迁移模型 13

2.3.3 LMS 持续校准 13

2.3.4 LLM 选择性跳跃 14

2.4 Preference 通道：约束惩罚自适应 14

2.4.1 动机 14

2.4.2 违规压力 14

2.4.3 不对称乘法更新 16

2.5 LLM 元控制器 16

2.5.1 动机 16

2.5.2 两个正交旋钮 16

2.5.3 上下文与证据要求 17

2.5.4 实现细节：模型、Prompt 与输出安全 18

2.5.5 与 LMS 的协同 19

2.6 Memory DB 与状态持久化 19

2.6.1 动机 19

2.6.2 表结构设计 19
2.6.3 跨轮智能 19

3 运行框架 22
3.1 系统基础设施 22
3.2 单次循环 Pipeline 22
3.3 YOLO 持续运行循环 24

4 评测 25
4.1 评测指标 25
4.2 实验设置 26
4.3 在线 A/B 结果 26
4.3.1 Exp-401838：A 国 PDP 热启动（7 轮） 26
4.4 离线-在线迁移分析 27
4.5 参数演化与稳定性 28
4.5.1 热启动实验中的残余钟摆效应 28
4.6 LLM 校准有效性 30
4.6.1 修正收敛 30
4.7 灵敏度分析 30
4.8 B 国部署：从冷启动到推全 32
4.8.1 冷启动信号 32
4.8.2 搜参历程：从方向到可部署参数 33
4.8.3 胜出参数结构 33
4.8.4 长周期 A/B 验证 34
4.9 效率与运营成本 34

5 讨论：一人之力的可行性与意义 37
5.1 交付物的完整范围 37
5.2 Sortify 库的技术深度与开发复杂性 37
5.3 从 ParaDance 到 Sortify：技术谱系与领域积淀 38
5.4 编排模式 40
5.5 元嵌套：智能体构建智能体 40
5.6 意义与影响 41

6 结论、局限与未来方向 43

A 贡献者 46

B 实现细节 46
B.1 A/B 实验集成 46

B.2 Redis 参数发布 46
B.3 状态文件管理 46
B.4 目录结构 47
C 符号表 47

1. 引言

推荐系统的排序决定了用户最先看到什么，直接影响交易额（GMV）、订单量、广告收入和用户留存等核心业务指标。工业级排序系统通常通过可调的打分函数和特征权重组合多路信号，尤其是在商品搜索和平台推荐场景中更是如此 Santu et al. [2017], Liu [2009]。排序参数优化因此成为持续且高杠杆的工程任务。随着平台向多市场、多业务线和多推荐场景扩展，对自动化、自修正参数优化能力的需求日益迫切 Hutter et al. [2019]。

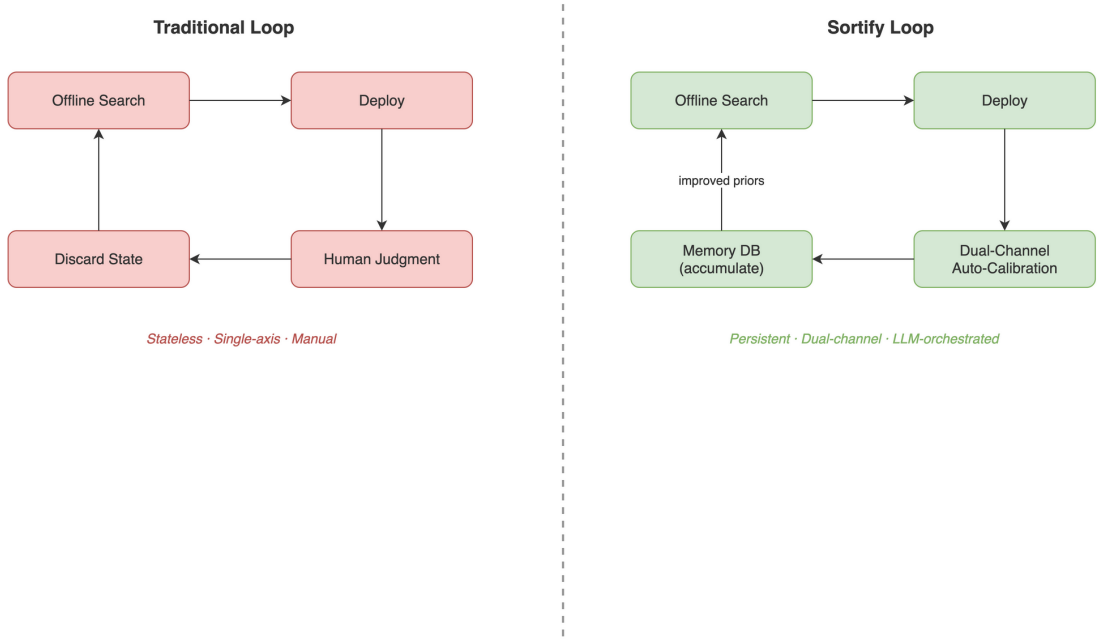

图 2 | 左——传统优化闭环：离线搜索 → 部署 → 人工判断 → 丢弃状态 → 重复。右——Sortify 闭环：离线搜索 → 部署 → 双通道自动校准 → Memory DB 积累 → 带改进先验重复。突出结构性差异：无状态 vs 持久化、单轴 vs 双通道、人工 vs LLM 编排。

当前主流的优化流程如下：离线搜索算法在日志数据上通过代理指标探索参数空间，找到候选配置后推送至生产环境进行 A/B 验证；运营人员审查 A/B 结果，人工判断是否采纳新参数或回退至旧配置 Gilotte et al. [2018]。这一人工闭环虽然概念清晰，却存在三个无法通过增量调优解决的结构性缺陷。

离线-在线迁移鸿沟。最关键的结构性问题在于，离线代理指标与在线业务效果之间存在系统性偏差。在保留于正文的 A 国热启动实验中，离线 I_{gmv} 从 +18.2% 提升到 +41.6%，对应的在线 GMV 却从 −3.6% 波动到 +9.2%，显示离线代理对在线收益存在显著乐观偏差。更复杂的是，同一轮参数更新对 GMV、Orders 和 Ads Revenue 的影响并不同步，说明不同业务指标的迁移关系并不共享单一的校准常数。标准的日志数据评估方法基于历史交互来估计候选策略价值 Gilotte et al. [2018], Dudik et al. [2014]，但它们本身并不会为非平稳生产环境提供持续更新、按指标细分的迁移校准。我们的生产观测表明，迁移函数会随流量模式、时段效应和竞争动态持续漂移。缺乏持续校准意味着，每一轮精心设计的离线搜索都建立在可能已经过时的映射假设之上，搜索精度带来的收益被迁移误差部分抵消。

诊断信号纠缠。当某轮优化在线结果不理想时，运营人员面临一个根本性的归因难题：是离线-在线映射预测失准（*Belief* 偏差），还是约束惩罚的敏感度设置不当（*Preference* 偏差）？两类失败模式要求截然相反的修正方向——Belief 偏差需要调整迁移函数截距，Preference 偏差需要重新缩放惩罚权重 Berger [1985]。在缺乏形式化分离的框架中，两种信号会被混为一谈：GMV 为正而 Ads Revenue 为负，既可能意味着映射乐观，也可能意味着约束太松。诊断信号纠缠不仅是调参精度问题，它还暴露了把多个目标和约束压缩到单一纠偏通道中的单体化优化流程局限 Lin et al. [2019]。

无持续学习能力。传统优化系统中，每一轮都从空白状态出发。第 N 轮学到的迁移关系不会传递给第 $N+1$ 轮；约束违反后发现的敏感度信息在下一次参数推送后即被遗忘。这种"土拨鼠之日"效应意味着，运行过 50 轮 A/B 实验的系统，其校准智能与第一轮完全相同 Parisi et al. [2019]。在我们的生产场景中，每个离线-在线观测对代表 3.5 小时的真实流量数据——获取成本非同小可。丢弃这些累积证据既是经济上的浪费，也是技术上的可避免损失。近期在持续学习 Parisi et al. [2019]、用于 warm start 的元学习优化 Feurer et al. [2015] 以及具备持久状态的 LLM Agent Wang et al. [2024] 方面的研究表明，跨 episode 的累积学习在序贯决策问题中既可行又有价值。

针对上述三个结构性问题，我们提出 Sortify——一个智能体驱动的闭环排序优化系统，将排序参数优化重构为持续的影响力置换过程。Sortify 不再将优化视为无状态的人工调参，而是让 LLM 智能体掌舵，在各排序因子间持续调控影响力的分配与兑换。Sortify 的核心架构创新在于，将影响力置换问题分解为两个正交通道——Belief（影响力预测如何映射到现实）和 Preference（影响力约束以多大力度执行）——并由 LLM 元控制器在框架级参数上进行编排，而非直接操作底层搜索参数。

Sortify 的设计基于三条原则。第一，*关注点分离*：迁移映射修正（Belief）和约束敏感度调整（Preference）在从贝叶斯决策论分解 $P(\theta|D) \times L(a,\theta)$ 推导出的独立轴上操作 Berger [1985]，确保一个轴上的修正不会干扰另一个轴。第二，*框架级 LLM 控制*：LLM 调整搜索框架——具体为迁移函数截距和惩罚乘数——而非 7 维排序参数向量本身。这一区分至关重要：框架参数编码的是可跨轮复用的经验（例如"过去 3 轮离线 GMV 预测持续偏乐观 5–8%"），而底层参数是绑定于单次数据快照的临时事实 Feurer et al. [2015]。第三，*累积记忆*：包含 7 张关系表的持久化 Memory DB 存储每一个离线-在线观测对、校准更新和 LLM 提案，使系统能够跨轮次积累校准智能，而非在每次迭代后重置 Parisi et al. [2019], Feurer et al. [2015]。

本报告的主要贡献如下：

- **双通道自适应框架**。我们提出 Belief/Preference 解耦方案，实现离线-在线迁移映射与约束敏感度的正交校准。在 A 国推荐平台的热启动生产 A/B 测试中，该框架在 7 轮内将 GMV 从 $\mathbf{-3.6\%}$ 推升至 $\mathbf{+9.2\%}$，订单量峰值达到 $\mathbf{+12.5\%}$（Section 4.3）。
- **基于证据推理的 LLM 元控制器**。我们引入一个 LLM Agent，通过两个框架级旋钮——迁移函数截距（$\delta_{\text{intercept}} \in [-0.1, +0.1]$）和惩罚乘数（$\in [0.5, 2.0]$）——基于 20 轮 episode 历史和 30 条校准轨迹进行调整。LLM 的校准工作量从每轮 5 项修正收敛至 2 项（7 轮内完成），显示框架级修正会随证据积累而下降（Section 4.6）。
- **持久化记忆架构实现累积学习**。我们展示，7 表关系型 Memory DB 能够让系统在既有校准

状态上热启动运行，并将跨轮学习积累延续到后续实验。保留于正文的 A 国案例直接以既有 Memory DB 状态启动，B 国案例则在无需架构修改的前提下完成跨市场部署（Section 4.5、4.8）。

本报告的结构安排如下。Section 2 介绍系统架构，详述 Influence Share 指标、双通道机制、LLM 元控制器和 Memory DB 的设计。Section 3 描述运行框架，包括 10 步单次循环 Pipeline 和 YOLO 持续运行循环的故障恢复机制。Section 4 提供来自 A 国和 B 国共 30 轮优化的评测证据，其中包括 7 轮 A 国热启动实验与 23 轮 B 国部署实验。Section 6 讨论局限性——包括残余参数振荡、低流量窗口统计不确定性以及极端断点实证不足——并展望未来方向。

2. 系统架构

本节完整介绍 Sortify 的系统架构——一个面向推荐系统排序参数持续优化的三层闭环系统。系统以在线 A/B 实验数据为输入，持续校准其对离线-在线迁移关系的内部模型，并输出优化后的排序参数用于下一部署周期。我们将分五个部分详述核心组件：Influence Share 指标与参数搜索引擎（Section 2.2）、Belief 通道的迁移校准机制（Section 2.3）、Preference 通道的约束自适应机制（Section 2.4）、LLM 元控制器（Section 2.5）和持久化 Memory DB（Section 2.6）。

2.1 总述

Sortify 采用三层架构，在人工配置、智能校准和参数搜索三个层面实现关注点分离。

- **第一层（外层 — 人工/配置）**：定义优化目标（如最大化 I_{gmv}）、约束边界（如 I_{order} 退化 $\leq 5\%$）、初始参数范围和惩罚权重种子。此层按市场设置一次，低频更新。
- **第二层（中间层 — LLM + 算法）**：执行双通道校准。Belief 通道通过 LMS 回归和 LLM 截距调整修正离线-在线迁移映射。Preference 通道通过乘法更新调整约束惩罚权重。LLM 元控制器利用累积 episode 历史编排两个通道。
- **第三层（内层 — Optuna TPE 搜索）**：在第二层产出的校准 target_range 和 penalty_weight 约束下，以 25 个并行工作线程执行 5,000 次试验，在 7 维参数空间中寻优。

这一三层架构中，第二层的双通道设计并非工程上的任意分割，而是基于理性决策公理的必然推演。L.J. Savage (1954) 的主观期望效用理论（Subjective Expected Utility, SEU）指出：**任何理性决策都需要且仅需要两个独立输入——你认为世界是怎样的（概率/信念），以及你在乎什么（效用/偏好）**。若一个系统的行为满足一致性公理，则其偏好可由主观概率与效用来表示；其中效用函数并非绝对唯一，而是只在正仿射变换下唯一。Sortify 将这一公理进行了操作化的系统分解：

- **Belief 通道（求真）**：对应 Savage 理论中的*状态信念*——“离线指标如何映射到在线现实？”它只关心客观物理与系统规律，不带任何价值判断。
- **Preference 通道（求善）**：对应 Savage 理论中的*效用函数*——“各约束被违反这件事，对我来说有多痛？”它只负责定义损失边界，不干涉物理规律。

如果这两者不被强制解耦，系统将陷入不可避免的认知偏差：**一厢情愿**（Wishful Thinking）——因为极其渴望目标达成，从而扭曲对离线-在线转化率的客观认知；或**酸葡萄心理**（Sour Grapes）——因为转化率不佳，为了让总体损失函数收敛，反而降低对红线约束的重视程度。双通道设计从架构级切断了这种“诊断纠缠”，确保系统在两个正交方向上独立自校准。

更进一步，一个智能体可以在这两个正交方向上犯截然不同的错误，且修正路径互斥：

1. **认知偏差（Epistemic Error）**：“我以为会发生，但并没有”——系统的世界模型失效。此时**绝不能修改惩罚权重**，必须修正信念映射。
2. **价值失调（Axiological Error）**：“发生了，但我没想到这么痛”——预测正确但效用评估不足。此时**绝不能修改迁移映射**，必须修正效用函数。

完整的数据流形成闭环：在线 A/B 数据作为新 episode 写入 Memory DB → LMS 更新迁移模型斜率和截距 → LLM 提出框架级修正 → 校准后的约束输入 Optuna 搜索 → 最优参数发

布至 Redis → 生成新的 A/B 数据 → 循环重复。每个周期约 4 小时完成。

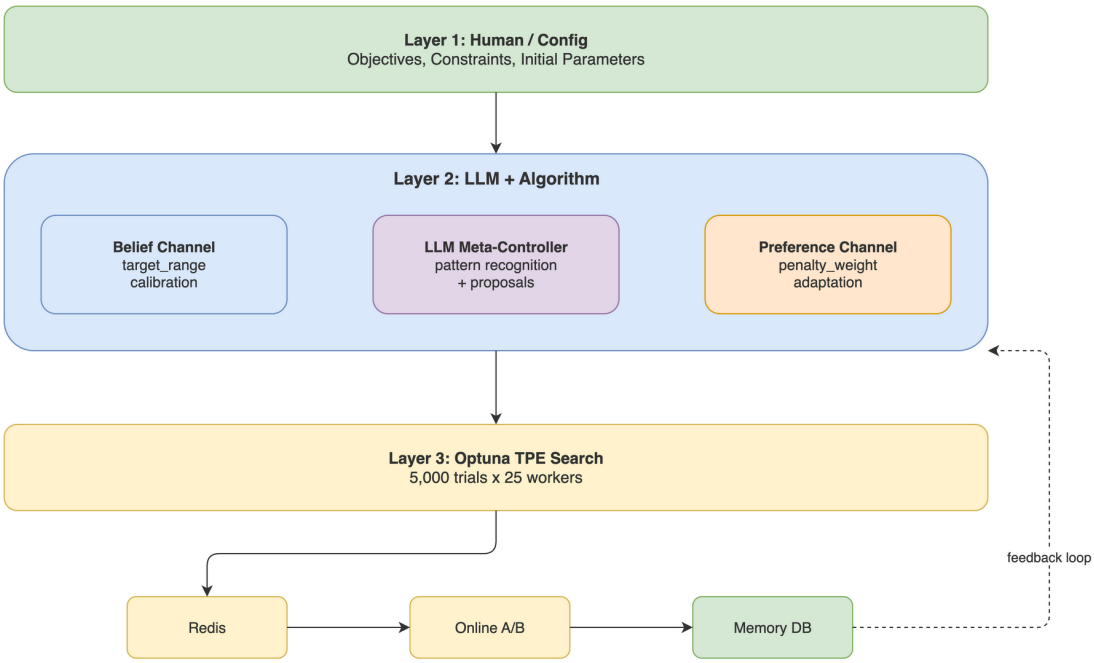

图 3 | 三层架构图。第一层（人工/配置）向第二层（LLM + 算法）输入目标和约束，第二层包含 Belief 通道、Preference 通道和 LLM 元控制器。第二层输出校准后的 target_range 和 penalty_weight 至第三层（Optuna TPE 搜索，5000 trials × 25 workers）。第三层产出最优参数 → Redis → 在线 A/B → Memory DB → 回到第二层。

2.2 Influence Share 与参数搜索引擎

物理直观：高维空间中的房间与墙

在深入探讨具体的数学推导和系统动机之前，我们先建立一个直观的几何物理图像，来理解什么是“排序”，以及什么是排序特征的“影响力”。

想象一次搜索请求召回了 n 个候选商品。在传统的工程视角下，排序就是给这 n 个商品各自计算一个总分，然后按分数高低从上到下排列。但如果我们将视角切换到高维几何空间，会得到一幅完全不同的画面。

我们可以把这 n 个商品的分数看作是 n 维空间中的一个“点”（即打分向量）。在这个广袤的空间里，存在着许多无形的“墙”。什么时候会碰到墙呢？当任意两个商品的分数完全相等时，这个临界状态就在空间中构成了一堵“超平面墙”（比较边界）。因为有 n 个商品，所以空间里横亘着无数面这样的墙。

这些墙相互交织，将整个 n 维空间切割成了无数个封闭的“房间”（在几何上称为 chamber）。每一个“房间”都代表着一种绝对确定的名次排列顺序。只要你的打分向量这个“点”安稳地待在某个房间内部，没有任何商品同分，那么最终的排序榜单就是确定且唯一的。

现在，我们要优化排序算法。在 Sortify 中，我们通过调整参数 $\boldsymbol{\theta}$（比如改变 GMV 的权重）来改变每个商品的最终得分。在几何图像中，调整参数的过程，本质上就是**推动这个代表整体打分的“点”在空间中移动**。

当参数发生变化，这个点开始移动。如果它只是在一个“房间”内部游走，排序名次根本不会发生任何变化（即便分数变了）。只有当这个点**穿过了一堵“墙”**，从一个房间跨入另一个相邻的房间时，才意味着有两个商品的分数发生了反转，名次互换了——此时，整体的排序结果才真正发生了改变。

这就是本系统的核心洞见：**排序改变的原子事件不是全局榜单的重排，而是打分点穿过某一面特定的墙。**

顺着这个思路，我们该如何定义各个业务因子（如订单量、GMV、广告）对排序的“**影响力”(Influence)**？

既然名次改变等价于“点撞破墙”，那么我们只需要进行一场物理上的“受力分析”。这堵墙的边界是由两个商品的分数差构成的，而总分数差是由各个因子单独的分数差累加而来的。因此，每个因子就像是一个在特定方向上施力的“推手”。各个因子的“影响力”，就是看在推动这个点穿过这堵墙的过程中，**每个因子在墙的法向方向上各自施加了多大的“推力”**。

Influence Share 这个指标，衡量的正是每个推手的力气占总推力的百分比。通过这种物理直观的映射，我们将抽象的参数调优和置换群里的排序变化，完美转化为了高维空间中清晰的受力与动能归因分析。这种视角让我们不再只盯着干瘪的排序相关性指标（如 Kendall Tau），而是能够深入到每一次名次交换的微观物理过程，去量化各个业务因子的真实贡献。

2.2.1 动机：超越 Kendall Tau

传统排序系统的参数搜索依赖 Kendall Tau 等排名相关性指标衡量候选参数对目标排序的保持程度。Kendall Tau 虽然能捕捉序数一致性，但在多因子排序优化场景中存在两个关键局限。第一，*不可分解*：Kendall Tau 产出单一标量，无法归因到具体因子，因而无法回答“排序决策中 GMV 贡献了多少、订单量贡献了多少”这类问题。第二，*不支持因子兑换分析*：由于 Kendall Tau 没有总和为一的约束，无法自然地表达“将 5% 的排序影响力从订单转移到 GMV，同时保持广告曝光稳定”这样的操作语义。

2.2.2 Influence Share 定义：从对称群到影响力置换算法

我们不将 Influence Share 视为一个孤立的经验性定义，而将其视为从排序的置换结构出发，对排序变化进行连续化归因的结果。设 request q 中有 n_q 个商品，记商品集合为 $X_q = \{1, \ldots, n_q\}$。在参数向量 $\boldsymbol{\theta}$ 下，该 request 的排序结果可表示为对称群中的一个置换：

$$\pi_q(\boldsymbol{\theta}) \in S_{n_q}. \tag{1}$$

这里 $\pi_q(\boldsymbol{\theta})(k)$ 表示排在第 k 位的商品。由于对称群 S_{n_q} 由相邻交换 $s_k = (k, k+1)$ 生成，任意两个排序之间的差异都可分解为一串局部 swap。因此，从控制论角度看，排序改变的原子事件不是“整个 permutation 被替换”，而是某一对商品的相对顺序发生翻转。

为把这一离散群结构与连续参数连接起来，令候选参数 $\boldsymbol{\theta}$ 在 request q 上产生总分向量 $\mathbf{S}_q(\boldsymbol{\theta}) \in \mathbb{R}^{n_q}$。对任意 $i \neq j$，定义比较超平面和对应 chamber：

$$H_{qij} := \{x \in \mathbb{R}^{n_q} : x_i = x_j\}, \qquad C_\pi := \{x : x_{\pi(1)} > x_{\pi(2)} > \cdots > x_{\pi(n_q)}\}. \tag{2}$$

当 $\mathbf{S}_q(\boldsymbol{\theta}) \in C_\pi$ 时，排序就是置换 π；当它穿过某个 H_{qij} 时，商品 i, j 的相对顺序翻转。这给出了对称群的经典反射几何表示。于是，排序变化的局部坐标就是 pairwise margin：

为避免记号歧义，我们默认总分处于 generic position；若出现并列，则由固定的 deterministic tie-breaker 打破，因此排序结果仍可视为 S_{n_q} 中的一个元素。

$$\Delta_q(i, j; \boldsymbol{\theta}) := S_q(i; \boldsymbol{\theta}) - S_q(j; \boldsymbol{\theta}). \tag{3}$$

在 Sortify 中，总分对因子是可加分解的：

$$S_q(i; \boldsymbol{\theta}) = \sum_{f \in \mathcal{F}} \mathrm{score}_{q,f}(i; \boldsymbol{\theta}), \qquad \Delta_q(i, j; \boldsymbol{\theta}) = \sum_{f \in \mathcal{F}} \Delta_{q,f}(i, j; \boldsymbol{\theta}), \tag{4}$$

其中

$$\Delta_{q,f}(i, j; \boldsymbol{\theta}) := \mathrm{score}_{q,f}(i; \boldsymbol{\theta}) - \mathrm{score}_{q,f}(j; \boldsymbol{\theta}). \tag{5}$$

这一步完成了从群论到算法的关键桥接：参数调整并不直接操作一个离散置换，而是先改变每个 pair 在比较法向方向上的 margin；margin 的符号决定是否发生 swap；一连串 swap 共同生成新的 permutation。基于这一视角，我们将参数搜索解释为一种影响力置换算法：它在所有关键 pair 上，持续重分配各因子对潜在 swap 的作用份额。

为了将这种作用份额写成稳定的度量，我们对每个 pair 施加五个归因要求：局部性、方向不变性、非负性、份额和为一，以及在各分量同比例缩放下的不变性。在这些要求下，最简洁的选择是对各因子的绝对 margin 做 L_1 归一化。定义 pair (i, j) 上的总影响预算为：

$$Z_{qij}(\boldsymbol{\theta}) := \sum_{f \in \mathcal{F}} \left| \Delta_{q,f}(i, j; \boldsymbol{\theta}) \right|. \tag{6}$$

若 $Z_{qij}(\boldsymbol{\theta}) = 0$，则所有因子在该 pair 上都不给出区分信号，该 pair 不携带排序信息，可从聚合集合中剔除。对所有 informative pair，因子 f 的成对影响力份额定义为：

$$s_{q,f}(i, j; \boldsymbol{\theta}) = \frac{|\Delta_{q,f}(i, j; \boldsymbol{\theta})|}{Z_{qij}(\boldsymbol{\theta})}. \tag{7}$$

由此立即得到 $\sum_{f \in \mathcal{F}} s_{q,f}(i, j; \boldsymbol{\theta}) = 1$。这里绝对值的作用，是将“方向”与“强度”分离：$\Delta_{q,f}$ 的符号决定因子把 pair 推向哪一侧，而 $|\Delta_{q,f}|$ 衡量该推动的强弱。在固定其他分量时，$s_{q,f}(i, j; \boldsymbol{\theta})$ 随 $|\Delta_{q,f}(i, j; \boldsymbol{\theta})|$ 单调增加，因此它提供了可解释的局部 influence signal，而不要求全局排序对参数变化呈严格单调。

从 pair 级份额到 request / 数据集级指标，我们只聚合 informative 且业务上重要的 pair。记 P_q 为 request q 的 pair 集，它可以是全部 informative pair，也可以限制在 Top-$(K + L)$ 区域。对 $(i, j) \in P_q$，令 r_{qi} 表示商品 i 在 request q 中的 rank，并定义位置敏感权重：

$$w_{qij} = \exp\Big(- \frac{\min(r_{qi}, r_{qj})}{\tau} \Big). \tag{8}$$

其中 τ 控制衰减速率。涉及更高排名位置的商品对获得更大权重，反映了页面顶部位置对业务影响不成比例地重要这一现实。

因子 f 的聚合 Influence Share 定义为：

$$I_f(\boldsymbol{\theta}) = \frac{\sum_q \sum_{(i,j)\in P_q} w_{qij} \cdot s_{q,f}(i,j;\boldsymbol{\theta})}{\sum_q \sum_{(i,j)\in P_q} w_{qij}}. \tag{9}$$

由构造保证 $\sum_{f\in\mathcal{F}} I_f(\boldsymbol{\theta}) = 1$。这一总和为一的性质正是实现“兑换”推理的基础：将 I_{gmv} 提升 5 个百分点，必然意味着其他因子合计减少 5 个百分点，使 trade-off 显式且可量化。至此，从群论对象到工程算法对象的推导闭合：S_{n_q} 中的排序由 pairwise swap 生成，swap 由比较超平面上的 margin 过零触发，而 Influence Share 则量化每个因子对这些潜在 swap 的作用份额。实现上，影响力置换算法可写成五步：计算各因子分数并求和得到总分；按总分排序得到 $\pi_q(\boldsymbol{\theta})$；在 P_q 上计算 $\Delta_{q,f}$ 与 $s_{q,f}$；按 w_{qij} 聚合；将 $I_f(\boldsymbol{\theta})$ 送入后续目标函数与约束。

2.2.3 多目标搜索建模

参数搜索优化一个复合目标函数，最大化主要业务指标（I_{gmv} 提升）同时通过二次惩罚约束违规：

$$\mathcal{J}(\boldsymbol{\theta}) = 10 \cdot \underbrace{\frac{I_{\text{gmv}}(\boldsymbol{\theta}) - I_{\text{gmv}}^{\text{base}}}{|I_{\text{gmv}}^{\text{base}}|}}_{\text{相对提升}} - \sum_{j=1}^{J} \lambda_j \cdot \left[\max(0,\ v_j(\boldsymbol{\theta}))\right]^2 \tag{10}$$

其中 $\boldsymbol{\theta} = (\text{ps_ads_wo}, \text{ps_ads_wg}, \text{ps_org_wo}, \text{ps_org_wg}, \text{ps_porg_w}, \text{ps_price_pow}, \text{ps_w2})$ 为 7 维参数向量，$v_j(\boldsymbol{\theta})$ 为第 j 个约束的违规量（违规时为正），λ_j 为对应约束的惩罚权重。

二次惩罚形式是经过深思熟虑的设计选择：

$$\text{penalty}_j = \lambda_j \cdot v_j^2 \tag{11}$$

1% 的违规产生 $\lambda_j \times 0.01^2$ 的惩罚，10% 的违规产生 $\lambda_j \times 0.10^2$ 的惩罚——两者相差 100 倍。这种凸性对轻微的边界触碰给予容忍，同时对大幅违规施加严厉打击，引导搜索偏好“所有约束略微满足”而非“一个约束严重违反”的方案。相比之下，线性惩罚仅将 10% 违规视为 1% 违规的 10 倍严重，对工业级约束执行力度不足。

约束涵盖 I_{order} 退化上限、$I_{\text{ecpm_term}}$ 退化上限、广告 Top-10 曝光率和 $I_{\text{gmv_ads}}$ 等边界条件。惩罚权重 λ_j 初始值设定在 15,000 至 30,000 之间，后续由 Preference 通道（Section 2.4）动态调整。

2.2.4 搜索引擎

搜索由 Optuna 的 TPE（Tree-structured Parzen Estimator）采样器执行 Akiba et al. [2019]，该方法对给定好/差目标值条件下的参数分布进行建模。每轮执行 **5,000 次试验**，使用 **25 个并行工作线程**配合 Constant Liar 策略——当某个工作线程在其他试验完成前启动新试验时，假设待完成试验将返回悲观值，从而避免冗余探索。典型搜索在 **15~30 分钟**内完成。

如 Section 4.3 所示，该搜索引擎在校准约束下持续找到使 I_{gmv} 相对基线提升 +4.9% 至 +41.6% 的参数配置。

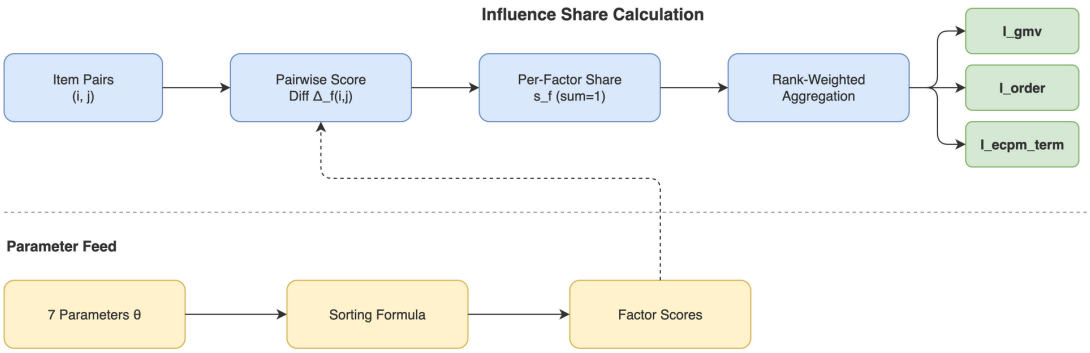

图 4 | Influence Share 计算流程。流程图：商品对 → 成对分数差 → 各因子份额（和 =1）→ 排名加权聚合 → I_{gmv}、I_{order}、$I_{\text{ecpm_term}}$。下方：7 个参数输入排序公式，改变各因子贡献。

2.3 Belief 通道：离线-在线迁移校准

2.3.1 动机

Section 1 记录的离线-在线迁移 Gap 并非固定常数——它在不同指标间各异，随时间漂移，且在流量模式变化时会发生突变。静态校准（如“离线 GMV 预测始终乐观 3 倍”）在数日内即会过时。Belief 通道通过为每个指标对维护一个持续更新的线性迁移模型来解决这一问题，该模型结合了慢速算法估计器（LMS）和快速模式识别 Agent（LLM）。

从贝叶斯认识论的角度看，Belief 通道内部展现了对不确定性世界的两种截然不同的应对哲学——信念更新的双重节拍：**常规科学与平滑先验**（LMS）代表经典贝叶斯框架，假设世界是连续的（马尔可夫性），每轮新证据微调先验；**范式转换与黑天鹅**（LLM 截距跳跃）则是当底层分布发生断崖式改变时，果断识别结构性断点，抛弃旧先验，强行注入新先验。前者处理日常的分布微漂移，后者应对突发的范式颠覆——两者共同构成了对不确定性世界的完备回应。

2.3.2 线性迁移模型

对每个指标对（如 $I_{\text{gmv}} \to$ GMV、$I_{\text{order}} \to$ Orders），我们将离线到在线的关系建模为：

$$\hat{u}_{\text{online}} = \alpha \cdot u_{\text{offline}} + \beta \tag{12}$$

其中 u_{offline} 和 u_{online} 分别为观测到的离线和在线提升幅度，α（斜率）捕捉迁移率，β（截距）捕捉系统性偏差。系统维护 6 组这样的关系：GMV~I_{gmv}、Orders~I_{order}、Ads Revenue~$I_{\text{ecpm_term}}$ 及三个派生指标。

2.3.3 LMS 持续校准

每轮结束后，斜率和截距通过学习率 $\eta = 0.2$ 的最小均方（LMS）回归更新：

$$\alpha_{t+1} = \alpha_t + \eta \cdot e_t \cdot u_{\text{offline},t} \tag{13}$$

$$\beta_{t+1} = \beta_t + \eta \cdot e_t \tag{14}$$

其中预测误差 $e_t = u_{\text{online},t} - \hat{u}_{\text{online},t}$。LMS 为渐进的分布漂移提供稳定、单调的收敛。然而，在突发的结构性变化（如促销活动或流量来源切换）下，$\eta = 0.2$ 的 LMS 大约需要 **15 轮**才能达到 95% 收敛——相当于超过 2 天的生产数据，在此期间系统使用失准的迁移模型运行。

2.3.4 LLM 选择性跳跃

为压缩结构性断点后的恢复时间，LLM 元控制器（Section 2.5）可以直接以离散跳跃方式调整截距 β，幅度最大 ± 0.1：

$$\beta_{\text{new}} = \beta_{\text{old}} + \Delta\beta_{\text{LLM}}, \quad \Delta\beta_{\text{LLM}} \in \{0, \pm 0.05, \pm 0.1\} \tag{15}$$

LLM 审查 20 轮 episode 历史，能够检测到单观测 LMS 无法察觉的模式——例如"连续 5 轮 GMV 迁移呈悲观偏差，尽管 I_{gmv} 趋势向好"。当 LLM 检测到此类模式时，单次截距跳跃即可实现 LMS 需要 10+ 轮才能达到的修正效果。关键设计是 LLM *仅调整截距*而非斜率，因为截距变化代表系统性偏差的漂移，而斜率变化意味着基本关系的结构性重构，后者应交由更保守的 LMS 估计器处理。

Belief 通道的输出是每个受约束指标的校准后 **target_range**——经校准迁移函数映射后对应可接受在线结果的离线提升范围。

如 Section 4.4 所示，即使在热启动条件下，Belief 通道也必须随新观测持续更新：Exp-401838 的 R2 仍表现为明显的离线乐观偏差，而到 R7 时相同的离线正向信号已能够转化为显著正向的在线 GMV。

2.4 Preference 通道：约束惩罚自适应

2.4.1 动机

即使迁移模型已完美校准，搜索框架仍需知道对每个约束的*执行力度*应有多强。第 N 轮中被轻微触碰的约束，在第 $N+1$ 轮可能因市场条件变化而变为关键性约束。Preference 通道基于观测到的违规模式，持续调整约束惩罚权重。

在 Savage 的 SEU 框架中，Preference 通道处理的是一种与 Belief 通道所修正的认知偏差（Epistemic Error）截然不同的错误类型——**价值失调**（Axiological Error）："发生了，但我没想到这么痛。"系统的预测完全正确，但对结果的"痛感"（Utility Loss）评估严重不足。例如，系统准确预测了某指标会轻微下降，在线确实轻微下降，但这一微小的下降却触发了业务大盘的灾难性报警。此时，修正路径**绝不能碰迁移映射**——因为世界模型没有错——而必须修正效用函数：系统在学习"这条红线比预期坚硬得多，下次即使牺牲主目标也绝不能再触碰。"

2.4.2 违规压力

对每个约束 j，计算归一化违规压力：

$$p_j = \frac{v_j(\boldsymbol{\theta})}{v_j^{\text{threshold}}} \tag{16}$$

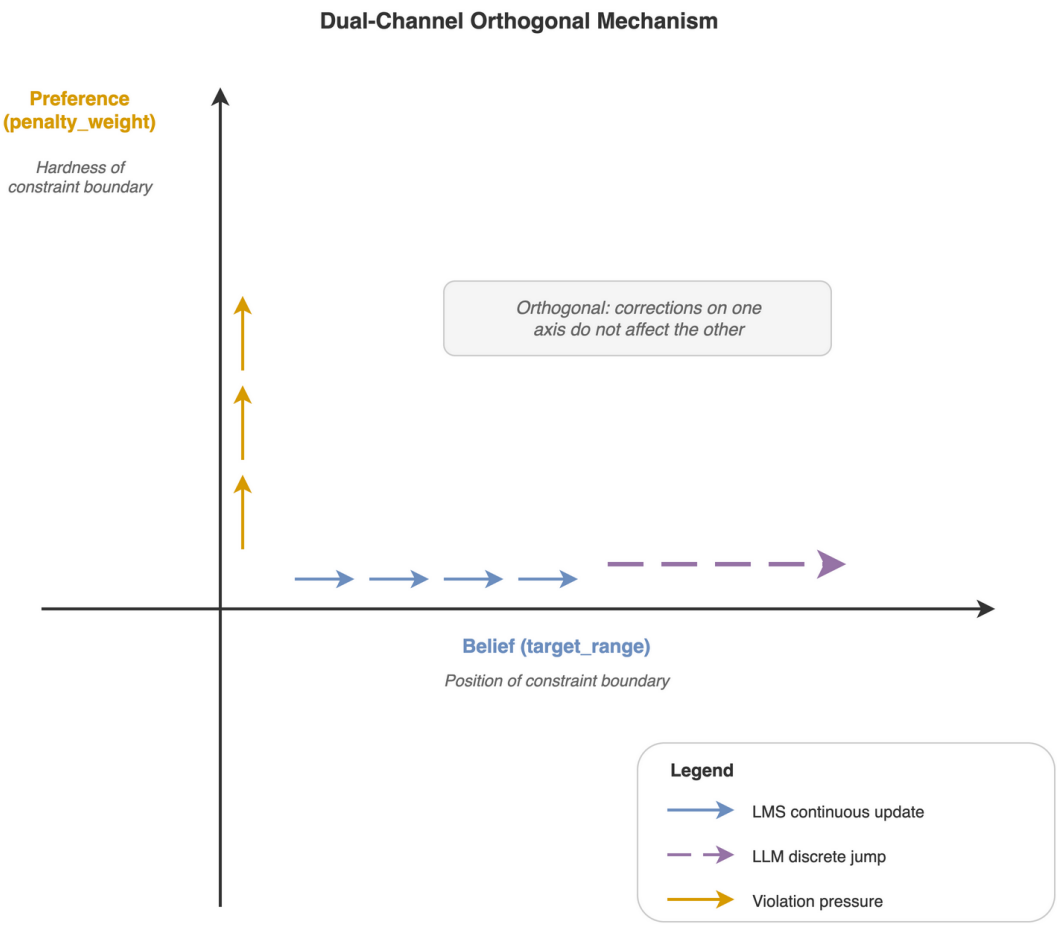

图 5 | 双通道机制。双轴图。水平轴：Belief（target_range = 约束边界的位置）。垂直轴：Preference（penalty_weight = 约束边界的硬度）。箭头显示 LMS 沿 Belief 轴连续移动，LLM 沿 Belief 轴做离散跳跃，违规压力沿 Preference 轴移动。两轴正交——一个轴上的修正不影响另一个轴。

其中 $v_j(\boldsymbol{\theta})$ 为观测到的违规幅度，$v_j^{\text{threshold}}$ 为可接受的违规阈值。$p_j > 0$ 表示约束被违反；$p_j \leq 0$ 表示约束被满足。

2.4.3 不对称乘法更新

惩罚权重更新遵循不对称步长的乘法规则：

$$\delta_j = \begin{cases} \delta_{\text{up}} = 0.25 & \text{if } p_j > 0 \\ -\delta_{\text{down}} = -0.08 & \text{if } p_j \leq 0 \end{cases} \tag{17}$$

$$\lambda_j^{(t+1)} = \lambda_j^{(t)} \cdot \exp(\delta_j) \tag{18}$$

这一设计编码了三个刻意的性质：

1. **尺度不变性**。乘法更新无论 λ_j 的绝对大小如何，施加相等的百分比变化。当一个约束 $\lambda = 1{,}000$ 而另一个 $\lambda = 80{,}000$ 时，相等的违规压力产生比例上相等的调整。加法更新会对小权重过度修正、对大权重修正不足。
2. **损失厌恶**。不对称性 $\delta_{\text{up}} = 0.25 > \delta_{\text{down}} = 0.08$ 意味着收紧约束（响应违规）比放松约束（满足时）**快 3 倍**。这反映了工业现实：约束违反的代价（如广告收入崩盘）远大于约束过紧的机会成本（如 GMV 略有欠优）。
3. **对数空间可解释性**。由于更新在原空间中是乘法、在对数空间中是加法，δ_{up} 和 δ_{down} 可直接解释为单轮收紧与放松的步长。二者刻意设置为不对称，使系统对约束违规更敏感、对约束放松更保守。

Preference 通道的输出是一组校准后的 **penalty_weight** 值，直接输入目标函数（公式 10）。如 Section 4.5 所示，Preference 通道的自适应权重有助于在后续实验中收窄参数振荡幅度。

2.5 LLM 元控制器

2.5.1 动机

Belief 通道中的 LMS 估计器能有效处理常规分布漂移，但从根本上说，它是一个单观测、固定步长的算法。它无法检测多轮模式（如“连续 5 轮 GMV 迁移悲观偏差”），无法区分噪声与结构性断点，也无法推理 Belief 修正与 Preference 修正之间的关系。LLM 元控制器通过提供 episode 级模式识别和跨通道推理来填补这一能力缺口。

从控制论（Cybernetics）框架来看，LLM 元控制器实现了从一阶控制向**二阶控制（反思）**的跨越。一阶控制——Optuna 搜索引擎在既定的 Belief 和 Preference 下于高维参数空间中寻找最优解——是对“环境”的直接控制。而 LLM 元控制器并不直接操控底层排序参数 $\boldsymbol{\theta}$，它的输入是“系统过去是如何看待数据的”（Memory DB 中的 LMS 更新历史和观测误差）——它在**学习“系统是如何学习的”**。这种对框架级元参数（截距、惩罚乘数）的干预，赋予了系统自省能力。LLM 完成的不是参数寻优，而是“信念的顿悟”与“价值观的重塑”。

2.5.2 两个正交旋钮

LLM 的操作严格限定在两个输出变量上：

$$\Delta\beta_{\text{LLM}} \in [-0.1, +0.1] \quad \text{（Belief 通道：截距调整）} \tag{19}$$

$$m_{\text{LLM}} \in [0.5, 2.0] \quad \text{（Preference 通道：惩罚乘数）} \tag{20}$$

这些边界是安全约束，不是建议值。单轮 ±0.1 的截距偏移已是相当大的修正；上限防止单次 LLM 错误导致灾难性的误校准。惩罚乘数 $[0.5, 2.0]$ 的范围允许将约束执行力度减半或翻倍，足以应对结构性断点，同时不会引发惩罚权重的失控膨胀。

一个关键设计决策是，LLM 调整的是*框架级*参数（截距和惩罚乘数），而非 7 维底层排序参数向量 $\boldsymbol{\theta}$。其原理植根于**经验**与**事实**的区分："过去 3 轮离线 GMV 预测持续偏乐观 5–8%"是一条可跨数据快照复用的结构性洞察，而"ps_ads_wg = 2.15"是关于今天数据分布的一次性事实，明天可能变成 1.73。LLM 在可复用经验的空间中操作；Optuna 在短暂事实的空间中操作。

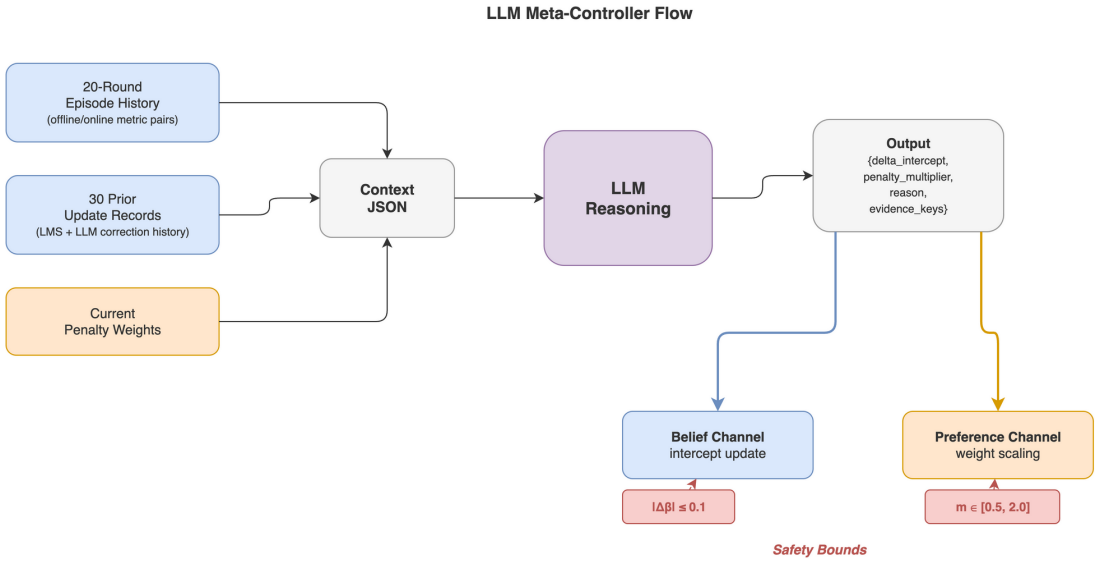

图 6 | LLM 元控制器。输入组装：20 轮 episode 历史（离线/在线指标对）+ 30 条 prior update 记录（LMS 和 LLM 修正历史）+ 当前惩罚权重 → Context JSON → LLM 推理 → 输出：{delta_intercept, penalty_multiplier, reason, evidence_keys}。两支箭头从输出分出：delta_intercept 输入 Belief 通道（截距更新），penalty_multiplier 输入 Preference 通道（权重缩放）。安全边界显示为两个输出的护栏。

2.5.3 上下文与证据要求

LLM 接收一个结构化的 context JSON，包含：(1) 全部 6 组 prior relation 的当前斜率和截距，(2) 最近 20 个 episode 的离线-在线配对指标，(3) 最近 30 条 prior update 历史记录（LMS 和 LLM 两类），(4) 当前惩罚权重状态。

每个 LLM 提案必须引用具体的 episode key 作为证据。缺乏证据的提案会被拒绝。当 LLM 置信度较低时——例如近期 episode 信号相互矛盾——LLM 被指示返回空提案（不调整），这优于一个依据薄弱的修正。

2.5.4 实现细节：模型、Prompt 与输出安全

模型与调用方式。 元控制器通过 Codex CLI[1] 的 `exec` 子命令以沙盒化子进程方式调用大语言模型。Sortify 自行构建了完整的记忆系统（Section 2.6）和工具调用机制，使其本身即为一个功能完备的自主 Agent——Codex 仅作为底层 LLM 推理引擎被调用，而非作为 Agent 框架使用。调用命令为：

```
codex exec - --skip-git-repo-check \
  --sandbox read-only --color never \
  -c model_reasoning_effort="high"
```

Prompt 通过 `stdin` 传入，模型响应从 `stdout` 捕获。`model_reasoning_effort="high"` 参数激活扩展推理模式，促使模型在生成结构化 JSON 输出前进行更深层次的思维链分析。超时阈值设为 300 秒以防止管道阻塞。未显式设定 Temperature 参数；扩展推理模式通过引导模型在给出答案前进行充分推理，隐式地替代了采样控制。

Prompt 结构。 LLM 接收一个单轮 Prompt，由两部分拼接而成：

1. **系统指令**（约 600 字，硬编码）：定义输出 JSON Schema（`proposal_id`，以及包含 `relation_key`、`delta_intercept`、`penalty_multiplier`、`reason`、`evidence_episode_keys` 的 `proposals` 数组），指定安全边界（$|\Delta\beta| \leq 0.1$, $m \in [0.5, 2.0]$），并将任务框架设定为贝叶斯决策对齐——将信念更新（迁移函数的截距修正）与偏好更新（约束违反的损失权重缩放）分离。
2. **Context JSON**（动态生成）：即 Section 2.5.3 所述的结构化上下文，由 `export-llm-context` 命令从 Memory DB 导出。

不提供 few-shot 示例。Prompt 刻意保持 zero-shot，以避免将模型锚定在过往修正模式上，从而抑制其发现新的结构性洞察的能力。

输出解析与安全层。 LLM 的原始文本输出在修改任何系统状态之前，需通过七层安全管线：

1. **Prompt 级指令约束**：系统指令明确约束输出格式和数值范围。
2. **鲁棒 JSON 提取**：三策略解析器依次尝试直接 `json.loads`、正则匹配 Markdown 代码块提取、花括号深度状态机——从可能含噪的输出中隔离出第一个完整 JSON 对象。
3. **Schema 验证与类型强转**：对每个 proposal 字段进行验证：缺失的 `delta_intercept` 默认为 0.0，缺失的 `penalty_multiplier` 默认为 1.0，所有数值字段强制转换为 `float`。畸形条目被静默丢弃。
4. **Relation Key 白名单校验**：提案中每个 `relation_key` 都与上下文中 prior relations 的键集合进行比对，未知键触发立即拒绝。
5. **硬编码 Clamp**：无论 LLM 输出什么数值，$\Delta\beta_{\text{LLM}}$ 在代码层被截断至 $[-0.1, +0.1]$，m_{LLM} 被截断至 $[0.5, 2.0]$。

[1] OpenAI Codex CLI, https://github.com/openai/codex

6. **全局权重边界**：即使经过乘法缩放，惩罚权重仍受配置定义的上下界约束（如 [1,000, 80,000]），防止跨轮累积性漂移。
7. **失败回退**：若 LLM 调用超时、返回不可解析的输出或产出空 proposals 数组，系统回退到空操作提案（$\Delta\beta = 0$, $m = 1.0$）。管道继续运行不受中断。

这一纵深防御设计确保 LLM 是一个*受界限约束的顾问*（bounded advisor）：它能通过结构性洞察加速收敛，但即便在对抗性或退化输出条件下也无法扰乱系统稳定性。

2.5.5 与 LMS 的协同

LMS 和 LLM 在每轮内按串行顺序执行：LMS 先更新（Pipeline 第 5 步），然后 LLM 读取 LMS 更新后的状态并提出调整（第 7–8 步）。这一顺序确保 LLM 在最新的算法校准基础上工作。为防止通道间振荡，设有冻结规则：当 Belief 通道更新超过阈值幅度时，Preference 通道冻结一轮，以保持诊断清晰度。

如 Section 4.6 所示，在 Exp-401838 中 LLM 校准工作量从每轮 5 项修正收敛至 2 项（7 轮内完成），说明框架级修正会随着证据积累而逐步减少。

2.6 Memory DB 与状态持久化

2.6.1 动机

缺乏持久状态时，系统将遭受“土拨鼠之日”效应：每一轮都在对过往迁移关系、惩罚敏感度和 LLM 修正模式毫无认知的状态下重新开始。运行 30 轮后，系统的校准智能与第 1 轮完全相同。Memory DB 通过提供一个关系型存储来解决这一问题，它累积系统生命周期中的每一个观测、校准更新和决策记录。

2.6.2 表结构设计

Memory DB 由 7 张 SQLite 表组成，设计目标是幂等写入和不可变审计：

该模式强制执行两个关键属性：

1. **幂等性**。`prior_observations` 中的复合键（relation_key + episode_key）防止管道步骤在故障重试时产生重复写入。同一观测写入两次，系统状态完全相同。
2. **不可变性**。`memory_events` 表充当预写日志，行永不被删除或修改，为调试校准异常提供完整的取证轨迹。

2.6.3 跨轮智能

Memory DB 支撑三种形式的跨轮智能：

- **LLM 上下文组装**。最近 20 个 episode 和 30 条更新历史记录被提取并格式化为 LLM 的输入上下文。随着数据库增长，LLM 获得越来越丰富的模式检测证据。
- **跨实验热启动**。 Exp-401838 以既有 Memory DB 状态热启动，直接复用了先前学习到的迁移模型和惩罚权重，而非以默认斜率和截距重新开始。这一能力使系统能够把过去 episode 的校准信息延续到新的生产轮次（Section 4.3、4.5）。
- **跨市场迁移**。部署到 B 国时，Memory DB 的表结构无需修改即可复用。系统以默认先验初

表 1 | Memory DB 表结构概览。

表名	职责	关键字段	写入模式
`episodes`	事实表：每轮优化一行，含离线-在线配对快照	episode_key, offline_metrics, online_uplifts, status	仅追加；状态流转
`offline_candidates`	每轮 Top-5 参数候选	episode_key, rank, params, metrics	按轮追加
`prior_relations`	Belief 通道当前状态：各指标对的斜率和截距	relation_key, slope, intercept, eta	按轮 Upsert
`prior_observations`	各指标对和 episode 的原始三元组	relation_key, episode_key, offline_uplift, online_uplift	带幂等约束追加
`prior_update_history`	Belief 通道审计轨迹	method (`lms`/`llm`), 旧/新斜率, 旧/新截距	仅追加
`penalty_wt_upd_hist`	Preference 通道审计轨迹	constraint_metric, pressure, step, 旧/新权重	仅追加
`memory_events`	不可变全局审计日志	event_type, idempotency_key, metadata_json	仅追加

始化，但在单轮内即积累了 B 国特异性的校准信息，正确识别出基线公式中有机点击权重（ps_org_wc）被严重低估（Section 4.8）。

如 Section 4.8 所示，持久化 Memory DB 使 Sortify 无需架构修改即可跨市场泛化，验证了校准智能应为累积型而非短暂型的设计原则。

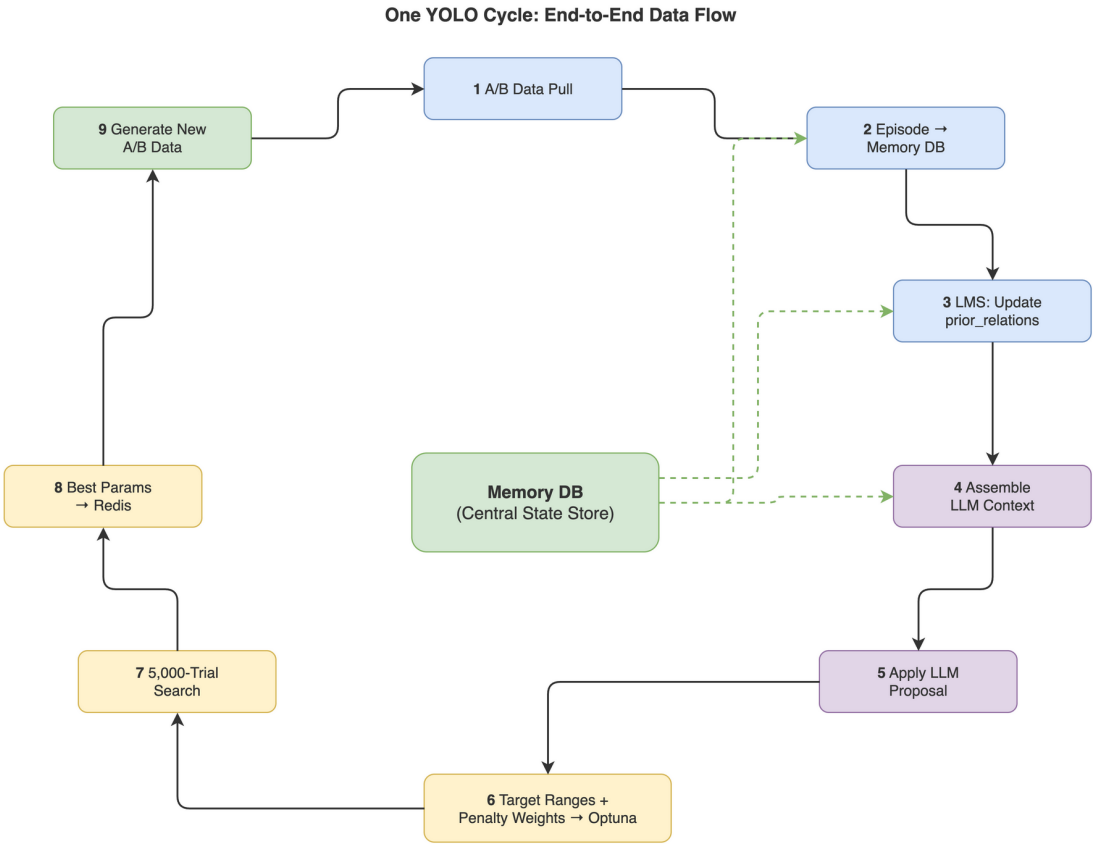

图 7 | 端到端数据流。一个完整 YOLO 周期的数据流：(1) A/B 数据拉取 → (2) Episode 写入 Memory DB → (3) LMS 读取 prior_observations，更新 prior_relations → (4) 从 episodes + update histories 组装 LLM 上下文 → (5) LLM 提案应用于截距和惩罚权重 → (6) Target ranges 和 penalty weights 输入 Optuna 目标函数 → (7) 5000 次试验搜索 → (8) 最优参数发布至 Redis → (9) 生成新 A/B 数据 → 回到 (1)。Memory DB 是连接所有步骤的中心状态存储。

3. 运行框架

本节描述 Sortify 作为生产系统的运行方式——运行所需的基础设施、单次优化轮次的逐步流程，以及将各轮串联为自主、自恢复过程的持续循环机制。目标是证明第 2 节所述架构不仅是理论框架，更是一个能够在内置故障处理下无人值守运行的可操作系统。

3.1 系统基础设施

表 2 | 系统配置。

组件	规格
搜索引擎	Optuna TPE 采样器 Akiba et al. [2019]
每轮试验数	5,000
并行工作线程	25（Constant Liar 策略）
状态数据库	SQLite (`agent_memory.db`)
参数发布	Redis 键值存储
每轮搜索时间	15–30 分钟
数据积累窗口	3.5 小时（最小值）
每轮处理时间	25–50 分钟
总周期时间	~4 小时
日吞吐量	~6 轮/天
时间对齐	15 分钟边界对齐

要点：总周期时间约 4 小时——3.5 小时在线数据积累加 25~50 分钟处理——使每天约可执行 6 轮优化。这一节奏足以响应日内流量模式变化，同时积累足够的数据量用于有统计意义的 A/B 比较。

系统的运行时资源占用刻意保持轻量化。Memory DB 是单个 SQLite 文件；搜索引擎在单台机器上运行；参数发布使用与其他生产服务共享的标准 Redis 实例，无需 GPU 资源。除平台现有 A/B 基础设施外，唯一的外部依赖是元控制器提案所需的 LLM API 调用。

3.2 单次循环 Pipeline

每轮优化执行一个 10 步流程。系统关注的是**可恢复性**而非逐次运行的严格确定性：持久化状态变更（如 episode 写入、prior 更新和状态文件更新）按幂等方式设计，因此若管道中断后重启，重复执行已完成的写入步骤不会破坏状态；但依赖外部服务的步骤，尤其是 LLM 提案生成，不假定逐次运行都会返回完全相同的结果。

步骤 1–4：数据采集。

- **步骤 1 — 拉取 A/B 报告**。获取当前观测窗口的在线 A/B 实验报告。报告包含 GMV、Orders、Ads Revenue、Advertiser Value、Organic Clicks 和 Ads CTR 相对于对照组的提升

百分比。

- **步骤 2 — 初始化 Memory DB**。若为首轮，创建 7 表结构。后续轮次验证表结构完整性并打开现有数据库。
- **步骤 3 — 记录离线指标**。将当前轮次的离线搜索结果（目标值、Influence Share 指标、参数向量、Top-5 候选）作为新 episode 写入 Memory DB，状态设为 `offline_recorded`。
- **步骤 4 — 记录在线指标**。解析步骤 1 的 A/B 报告，将在线提升值写入 episode 记录，并将 episode 状态流转至 `online_recorded`。

步骤 5：LMS 校准（Belief 通道） 对 6 组 prior relation 中的每一组，计算校准迁移模型的预测值与实际在线提升之间的预测误差，然后对斜率和截距应用 LMS 更新（Eq. 13、14）。所有更新持久化至 `prior_relations`，并以方法标签 `lms_regression` 记入 `prior_update_history`。

步骤 6：LLM 上下文导出 组装 LLM 上下文 JSON，提取：(1) 全部 6 组关系的当前斜率/截距，(2) 最近 20 个 episode，(3) 最近 30 条 prior update 历史记录，(4) 当前惩罚权重状态。将上下文写入带时间戳的文件以供审计。

步骤 7：LLM 提案生成 以结构化上下文和系统提示词调用 LLM API。LLM 返回 JSON 提案，包含 `delta_intercept` 条目（每个关系一条）和 `penalty_multiplier` 条目（每个约束一条），各附带 `reason` 字段和引用具体 episode 的 `evidence_keys`。当 LLM 置信度低时，返回空提案。

步骤 8：应用 LLM 提案 对每个非空提案项：将 `delta_intercept` 应用于 `prior_relations` 中对应关系的截距，将 `penalty_multiplier` 应用于对应约束的惩罚权重。所有变更记入 `prior_update_history`（方法标签 `llm_proposal`）和 `penalty_weight_update_history`。若 Belief 更新幅度超过冻结阈值，本轮跳过 Preference 更新。

步骤 9：推导 Target Range 使用更新后的迁移模型（斜率、截距），计算对每个受约束指标映射到可接受在线结果的离线提升范围。该校准后的 `target_range` 替换搜索目标中的静态约束边界。

步骤 10：推导 Penalty Weight 基于最近一个 episode 的结果计算每个约束的违规压力（Eq. 16），应用不对称乘法更新（Eq. 17、18），并持久化新的惩罚权重。

最终步：搜索与发布 以校准后的 target range 和 penalty weight 执行 Optuna TPE 搜索（5,000 次试验，25 个工作线程）。提取最优参数向量，发布至 Redis，并写入交接状态文件（`one-shot-latest.env`），记录运行标签、配置路径和最优参数，供下一轮使用。

3.3 YOLO 持续运行循环

YOLO（You Only Live Once）模式将单次循环 Pipeline 的执行串联为一个包含三阶段的无限自主循环。

阶段一 — 冷启动。 首次启动时，使用默认或用户指定的参数引导初始参数至 Redis。将观测时间窗口锚定至当前时间戳，对齐到 15 分钟边界。此时尚无 A/B 数据；跳过 LLM 诊断，以默认先验直接进入离线搜索。

阶段二 — 初始化。 执行首次完整的单次循环 Pipeline，支持可选的人工确认。该轮在 Memory DB 中建立基线 episode 和首个 A/B 观测对。

阶段三 — YOLO。 进入无限循环：

1. 等待至最小数据积累窗口（3.5 小时）从上次参数推送后已过。
2. 拉取已过窗口的 A/B 报告。
3. 执行完整的单次循环 Pipeline（步骤 1-10 + 搜索 + 发布）。
4. 更新 `yolo-state.env`：新的轮次编号、推送时间戳、观测窗口、上一组参数和 A/B 报告路径。
5. 向 `yolo-event-log.md` 追加完整轮次记录（轮次编号、时间戳、参数、A/B 结果、LLM 提案、错误）。
6. 返回步骤 1。

故障处理。 YOLO 循环区分两类故障：

- **A/B 数据获取失败**（瞬态）：以 5 分钟间隔重试最多 5 次。检测标准：A/B 报告文件缺失或少于 3 行。
- **非 A/B 故障**（搜索崩溃、Redis 错误、DB 损坏）：立即退出，不重试。运营人员排查后手动重启。

状态恢复。 崩溃或手动停止后重启时，YOLO 循环读取 `yolo-state.env` 恢复状态：最后完成的轮次编号、最后参数推送的时间戳、观测窗口边界和上一组参数向量。循环从下一轮继续，复用观测窗口终点作为新窗口起点。强制重置（`--reset` 标志）清除状态文件并从阶段一重新开始。

如第 4.3 节和第 4.8 节所示，YOLO 循环在 Exp-401838 中连续运行了 7 个轮次（25 小时），并在 B 国两个搜参阶段累计执行了 23 轮，验证了该设计的运行鲁棒性。

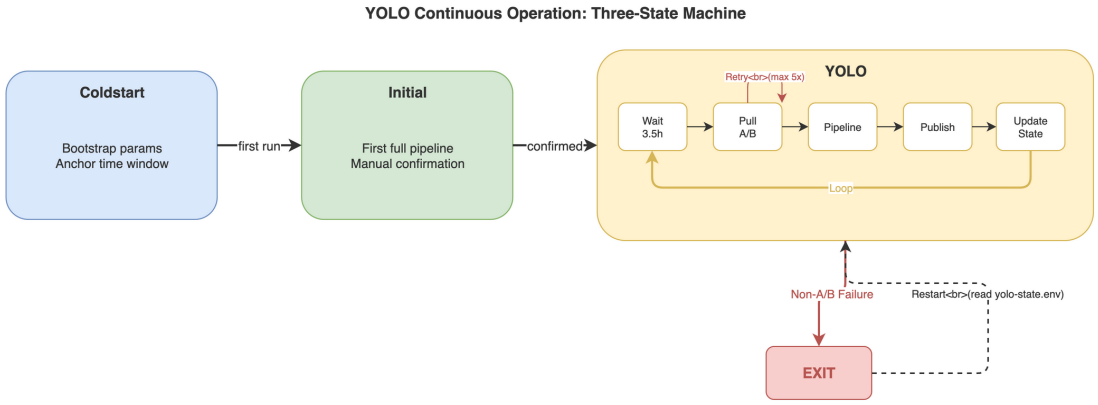

图 8 | YOLO 持续运行循环：三状态状态机图——冷启动（引导参数，锚定时间窗口）→ 初始化（首次完整 Pipeline，支持人工确认）→ YOLO（无限循环：等待 3.5h → 拉取 A/B → Pipeline → 发布 → 更新状态 → 循环）。重试逻辑显示为 A/B 拉取步骤上的自循环（最多 5 次重试）。非 A/B 故障的退出箭头。重启读取 yolo-state.env 回到 YOLO 状态的恢复箭头。

4. 评测

本节提供来自两个市场（A 国和 B 国）共 30 轮优化的实验与部署证据。我们从六个维度评估 Sortify：在线业务有效性（4.3 节）、离线-在线迁移校准质量（4.4 节）、参数稳定性（4.5 节）、LLM 元控制器有效性（4.6 节）、灵敏度分析（4.7 节）以及 B 国从冷启动到生产推全的完整部署案例（4.8 节），最后给出效率分析（4.9 节）。

4.1 评测指标

我们使用三类指标评估 Sortify：

离线指标（基于 Influence Share）：

- $I_{\mathbf{gmv}}$：GMV 因子在排序决策中的影响力份额。主要优化目标。
- $I_{\mathbf{order}}$：订单因子的影响力份额。受约束以防止订单过度退化。
- $I_{\mathbf{ecpm_term}}$：广告 eCPM 因子的影响力份额。受约束以保护广告收入。
- $I_{\mathbf{gmv_ads}}$：广告渠道内 GMV 的影响力份额。
- **ads_top10_rate**：广告出现在 Top-10 位置的比例。受约束以维持广告曝光。

在线指标（A/B 测试）：

- **GMV**：交易额相对对照组的提升。主要业务 KPI。
- **Orders**：订单量提升。次要业务 KPI。
- **Ads Revenue**：广告总收入提升。
- **Advertiser Value**：广告主 ROI 复合指标。
- **Organic Clicks**：自然（非广告）商品列表的点击量。
- **Ads CTR**：广告位的点击率。

衍生评测指标：

- **迁移率**：在线提升与离线提升的比值（如在线 GMV% / 离线 I_{gmv}%），衡量校准质量。

- **振荡比**：`ps_ads_wo`（最不稳定参数）在各轮中最大值与最小值之比，衡量参数稳定性。
- **LLM 修正数**：每次 LLM 提案中非零调整项的数量，衡量校准工作量收敛情况。

4.2 实验设置

表 3 | 实验概览。

	Exp-401838	**Exp-437160**
市场	A 国 PDP	B 国 PDP
轮次	7（热启动）	23（V1: 11 轮 + V3: 12 轮）
持续时间	25 小时	V1: 03-05–03-07; V3: 03-10–03-13; 冻结 03-14; 7 天 AB 03-15–03-22; 推全 03-24
启动条件	从既有 Memory DB 状态热启动	默认先验冷启动
观测窗口	每轮 3.5 小时	V3 每轮 6 小时；冻结后 7 天静态 AB
处理时间	25–50 分钟/轮	25–50 分钟/轮
参数	7 个（ps_ads_wo/wg, ps_org_wo/wg, ps_porg_w, ps_price_pow, ps_w2）	7 个（ps_org_wc/wg/wo, ps_ads_wc/wg/wo, ps_biz_price_pow）

Exp-401838 是一个保留了历史校准状态的 A 国热启动实验。其核心目的不是重新证明冷启动阶段会发生什么，而是检验一个已经积累了迁移模型和惩罚权重的系统，能否在生产环境中继续沿着更优方向推进，并在有限轮次内收敛 LLM 校准工作量。Exp-437160 则验证系统在未见市场中的端到端部署能力。该实验经历了两个搜参阶段——V1（11 轮，GMV 胜率 30%）暴露了冷启动在未见市场中的局限性，而 V3（12 轮）在 R7 识别出帕累托最优参数配置。该配置于 03-14 冻结，经过 7 天 A/B 验证后于 03-24 推全至生产环境。

两个实验在生命周期上承担不同角色。Exp-401838 展示了内层优化循环机制——搜参、部署、观测、校准——在热启动生产环境中的连续运行。Exp-437160 则将此延伸至完整的部署生命周期：它经历了探索、开发、参数冻结、长周期验证和生产推全全流程，证明系统能够产出可部署的策略，而不仅仅是有潜力的单轮信号。

4.3 在线 A/B 结果

4.3.1 Exp-401838：A 国 PDP 热启动（7 轮）

要点：Exp-401838 的 GMV 在 7 轮中总体向上推进，从 R2 的 **−3.6%** 提升到 R7 的 **+9.2%**；其中 R4 至 R7 连续 4 轮为正。Orders 自 R3 起持续为正，峰值达 **+12.5%**。这表明在热启动条件下，系统能够把已有校准状态延续到后续轮次，而不是在每轮重新从空白开始。

注意事项。 R7 指标采集于低流量的凌晨窗口（约 12K impressions），统计不确定性高于典型观测窗口。+9.2% GMV 和 +12.5% Orders 的结果需在高流量时段验证。

表 4 | Exp-401838 在线 A/B 结果——逐轮 GMV 与 Orders 进展。

轮次	GMV	Orders	Ads Revenue	Advertiser Value
R2	−3.6%	0.0%	+0.7%	+7.9%
R3	−1.5%	+1.7%	−0.8%	−1.0%
R4	+0.2%	+1.7%	−8.7%	−8.1%
R5	+1.9%	+3.8%	−5.8%	−5.6%
R6	+0.9%	+2.7%	−9.4%	−10.8%
R7	**+9.2%**	**+12.5%**	+5.7%	−8.9%

广告主价值持续承压。 Advertiser Value 仅在 R2 为正（+7.9%），后续轮次均转为负值，表明即使在热启动条件下，对 GMV 和 Orders 的优化仍可能系统性地压缩广告主 ROI——这一局限将在 Section 6 讨论。

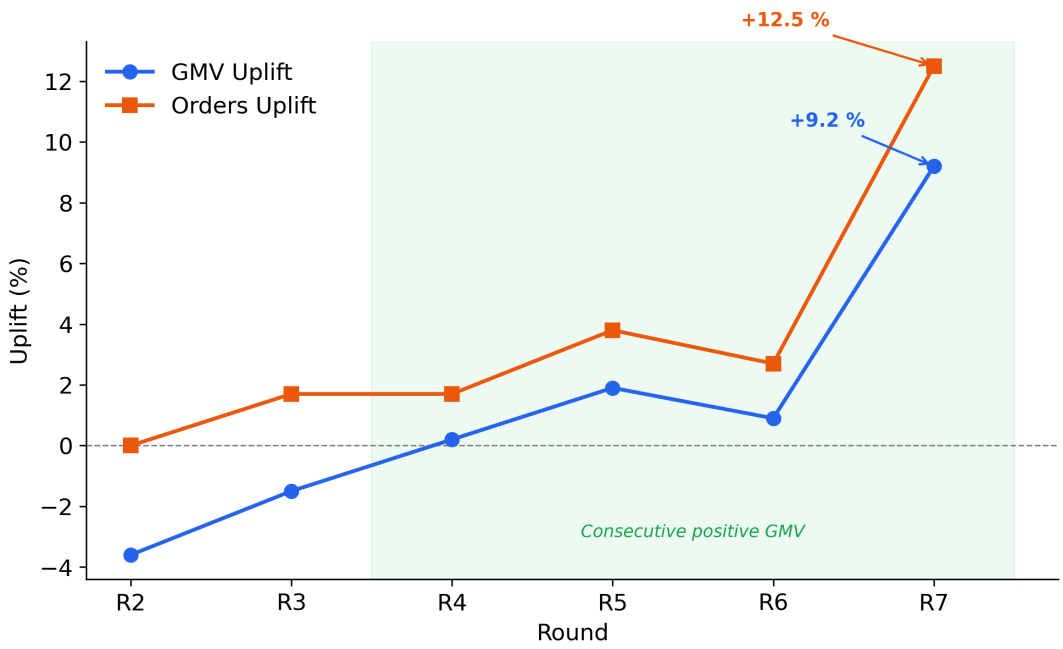

图 9 | 在线 KPI 趋势：单面板折线图，展示 Exp-401838 的 GMV 和 Orders 提升随 7 轮变化。虚线水平线标注 0% 参考。图中突出 R4–R7 的连续正向区间，以及 R7 的峰值结果。

4.4 离线-在线迁移分析

Sortify 的核心主张之一是：即使在热启动条件下，离线-在线映射仍然需要持续校准。Exp-401838 本身就给出了两个有代表性的配对观测。

要点：同一个热启动实验内部，离线-在线映射就会在轮次间发生明显变化。R2 说明“离线为正”并不足以保证在线为正；R7 则说明经过多轮校准后，离线收益终于开始更可靠地转化为在线 GMV。这一现象支持了 Belief 通道的必要性：迁移关系不是一次性设定，而是需要随新观测

表 5 | Exp-401838 中的代表性离线-在线 Gap。

轮次	I_{gmv} 提升	在线 GMV	观察
R2	+18.2%	−3.6%	离线显著乐观；正向离线提升并未转化为正向 GMV
R7	+41.6%	+9.2%	仍存在乐观偏差，但校准后已可转化为显著正向在线收益

持续更新。

进一步看 Section 4.3 中的其他指标，GMV、Orders、Ads Revenue 和 Advertiser Value 并不同步移动。例如 R7 同时出现 GMV +9.2%、Orders +12.5%、Ads Revenue +5.7%，但 Advertiser Value 仍为 −8.9%。这说明不同业务指标不应共享单一全局校准因子，而应维护各自的迁移关系。

4.5 参数演化与稳定性

4.5.1 热启动实验中的残余钟摆效应

所有实验中最不稳定的参数是 `ps_ads_wo`（广告订单权重），它控制广告可见性与自然体验之间的 trade-off。该参数表现出一种我们称为“钟摆效应”的特征性振荡模式。

表 6 | Exp-401838 参数振荡统计。

统计量	Exp-401838
`ps_ads_wo` 范围	1.96 – 8.91
振荡比（最大/最小）	**4.5 倍**
完整摆动周期	2
第二周期窄于第一周期	是

要点：即使在热启动条件下，`ps_ads_wo` 仍然是最不稳定的参数，但其摆动已被限制在 **4.5 倍**的范围内，且第二个周期窄于第一个周期。这说明系统尚未完全收敛到单一稳定操作点，但已经形成了受约束的残余振荡，而非无界漂移。

根因分析。 钟摆效应源于排序公式中的根本性 trade-off：提高广告可见性（高 `ps_ads_wo`）能改善广告曝光和潜在广告收入，但减少自然点击曝光并可能压制自然 GMV。当搜索引擎在某轮找到高广告权重方案最大化 I_{gmv} 时，在线结果揭示了自然体验退化，导致下一轮校准后的约束惩罚广告权重——将参数摆向相反极端。这种振荡反映了一个真实的多目标张力，单目标加惩罚的建模方式只能近似而非消解。

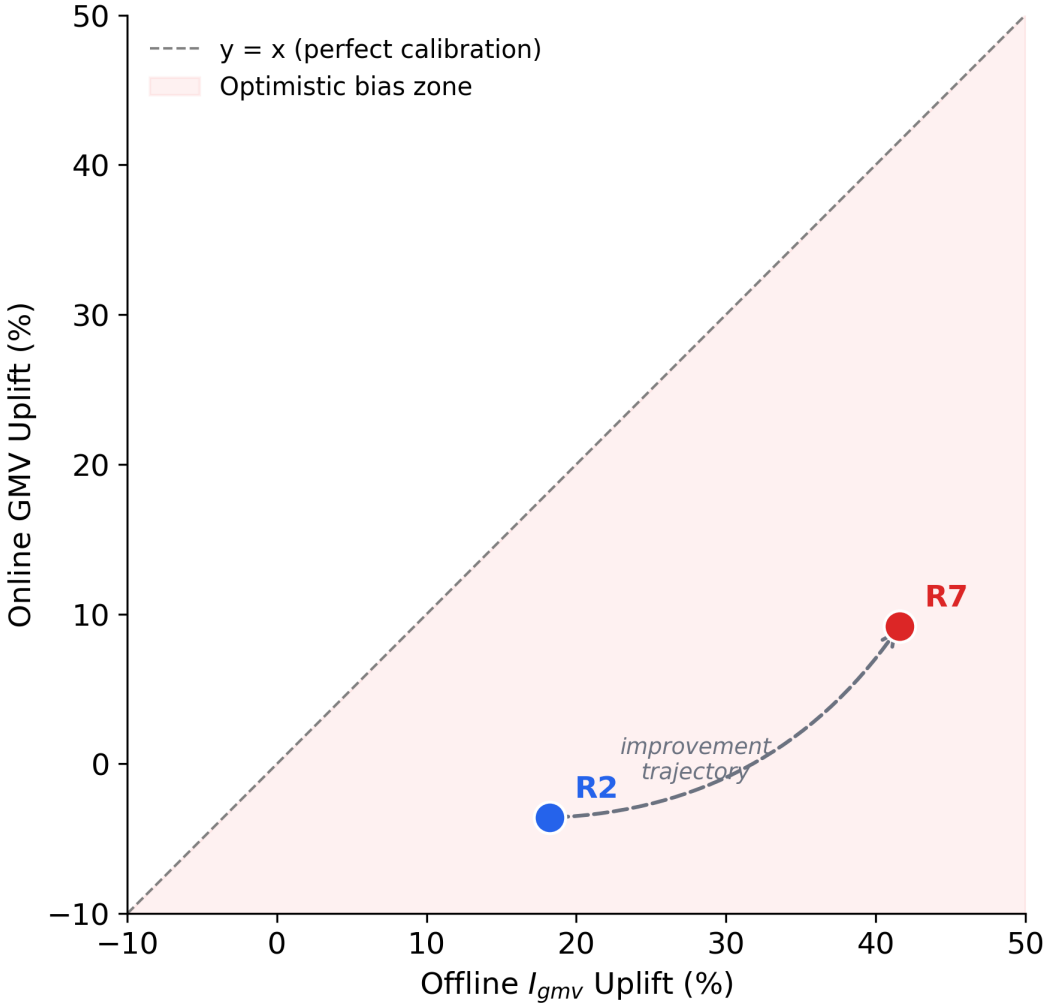

图 10 | 离线-在线迁移 Gap 可视化：散点图，横轴为 Exp-401838 各轮的离线 I_{gmv} 提升，纵轴为对应在线 GMV。恒等线（$y = x$）作为参考。点整体落在恒等线下方，展示系统性乐观偏差；后期轮次更接近正向象限。

表 7 | Exp-401838 的 LLM 修正收敛情况。

阶段	Exp-401838
早期（R2）	5 项，全面重校准
中期（R3–R6）	2–3 项，幅度递减
后期（R7）	2 项，最小幅度（−0.005）
收敛趋势	**清晰收敛**：5 → 2 项，幅度趋近 0

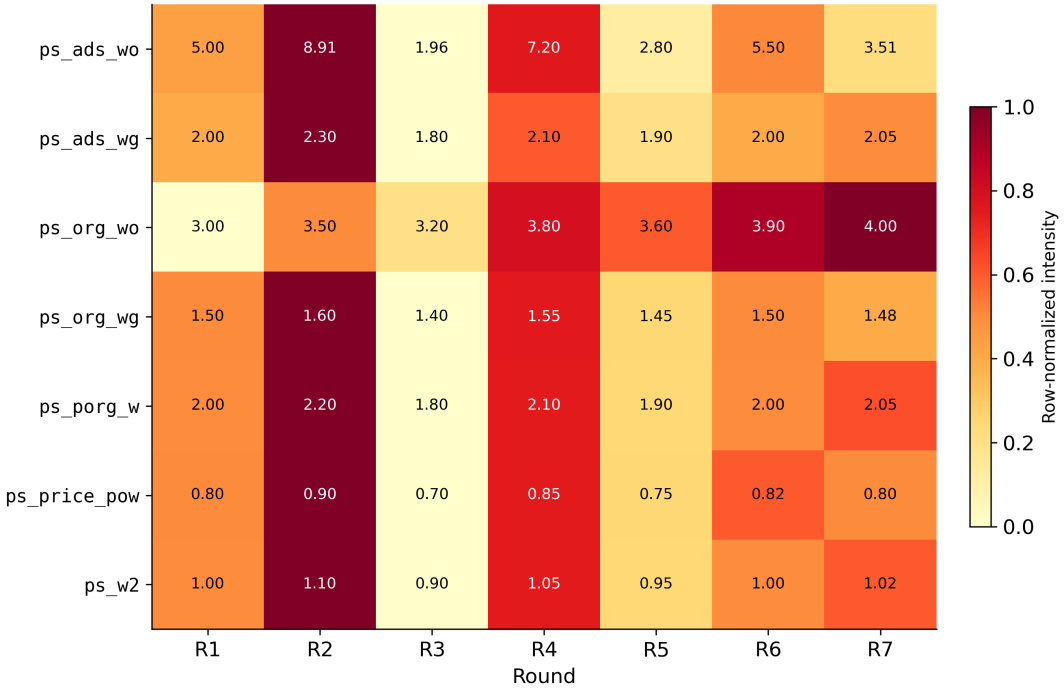

图 11 | 参数演化热力图：7 个参数（行）× 7 轮（列），数据来自 Exp-401838。色阶从低值到高值。`ps_ads_wo` 行显示最明显的往返摆动，但后半程波动区间收窄。

4.6 LLM 校准有效性

4.6.1 修正收敛

要点：在 Exp-401838 中，LLM 校准工作量从第 2 轮的 5 项修正收敛至第 7 轮的 2 项，且后期幅度趋近于零。这表明在热启动条件下，LLM 的角色更接近“补足剩余校准误差”，而不是持续重写整个迁移框架。

4.7 灵敏度分析

本节讨论系统关键超参数的选择依据及其对性能的影响。

LMS 学习率 $\eta = 0.2$。 学习率控制 Belief 通道对单个观测的响应速度。$\eta = 0.2$ 意味着每轮仅修正预测残差的 20%，从任意初始偏差收敛到 95% 精度需要约 $\lceil \log(0.05)/\log(0.8) \rceil = 14$ 轮。更高的 η 可加速收敛但增加对噪声观测的敏感性——在低流量窗口（如 Exp-401838 R7 的约 12K impressions），单轮观测的方差较大，过高的学习率会导致 slope/intercept 在噪声驱动下不必要地振荡。$\eta = 0.2$ 是一个经典的自适应滤波器取值，在收敛速度和噪声鲁棒性之间取得了合理平衡，且 LLM 的选择性跳跃机制弥补了面对结构性断点时 LMS 响应过慢的不足。

惩罚不对称性（$\delta_{\text{up}} = 0.25$ vs $\delta_{\text{down}} = 0.08$）。 收紧步长约为放松步长的 3 倍，编码了工业场景中“约束违反的代价远大于约束过紧的机会成本”这一非对称风险偏好。若将两者设为对称（如均为 0.15），惩罚权重将更容易在约束满足后快速衰减，增加后续轮次再次违反的概率——在

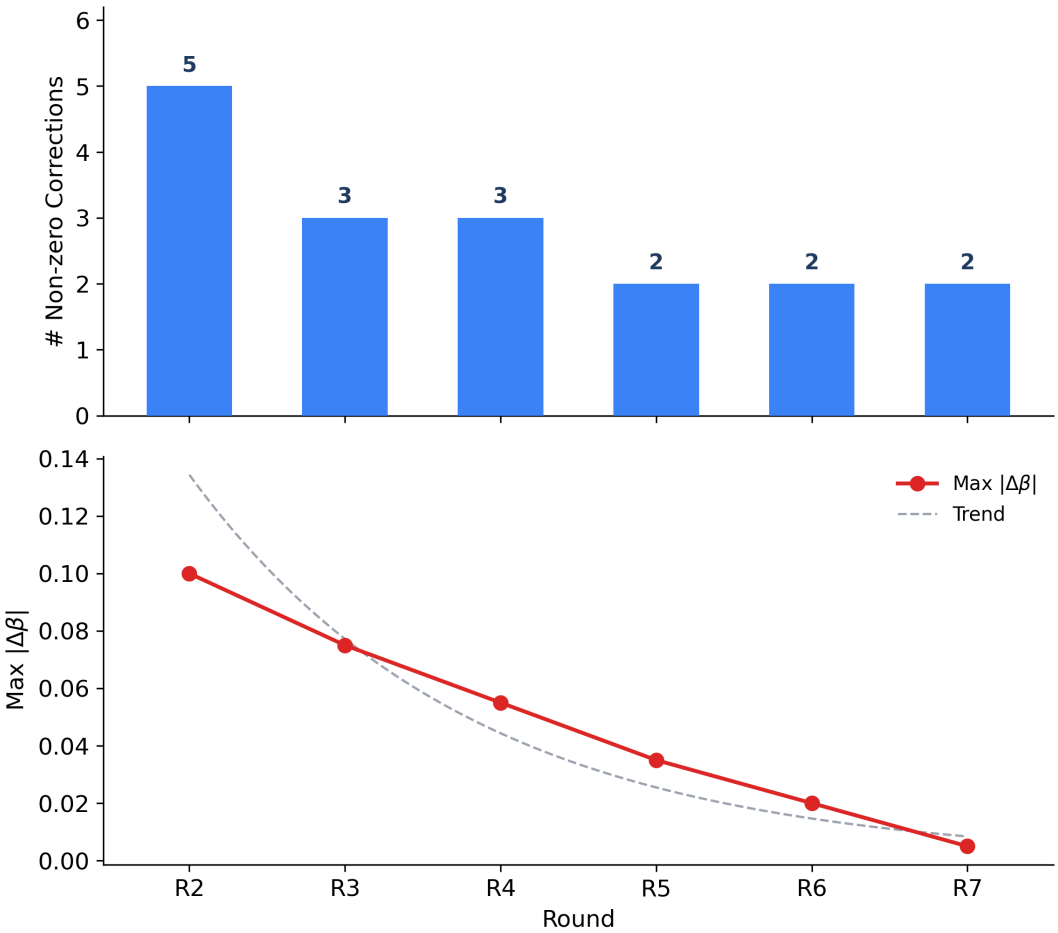

图 12 | LLM 修正收敛：双面板可视化。上面板：柱状图显示 Exp-401838 各轮非零 LLM 修正项数，从 R2 的 5 项逐步下降到 R7 的 2 项。下面板：折线图显示每轮最大修正幅度，整体呈递减趋势。

A/B 测试中，每次约束违反（如广告收入崩盘）都可能触发人工干预甚至实验终止，其代价远超 GMV 优化上偶尔的保守损失。

LLM 安全边界 ($|\Delta\beta| \leq 0.1$, $m \in [0.5, 2.0]$)。 安全边界确保即使 LLM 判断完全错误，单轮造成的偏移也在可控范围内。以截距为例，±0.1 的单轮最大偏移在 6 组关系中意味着 `target_range` 的最大单轮位移约为 10 个百分点——足够大以覆盖实务中数个百分点级别的系统性偏差，但不至于在误判时造成灾难性的搜索框架扭曲。惩罚乘数 $[0.5, 2.0]$ 允许单轮内将约束执行力度减半或翻倍，足以响应突发的约束违反模式。

冻结机制的必要性。 当 Belief 通道出现一次较大的更新时，若同时允许 Preference 通道大幅调整惩罚权重，那么下一轮在线结果将难以归因——很难判断 GMV 的变化究竟来自映射修正还是约束强度改变。冻结一轮的目的，是把一次显著校准的影响隔离出来，为后续诊断提供更清晰的信号。

惩罚权重 clamp 范围 $[1{,}000, 80{,}000]$ 的选择依据。 下界 1,000 的设定考虑了目标函数中正向收益的量级：典型的正向得分（$10 \times I_{\text{gmv}}$ 相对提升）在 0.5~2.0 的范围内（对应 I_{gmv} 提升 5%~20%），若惩罚权重低于 1,000，即使 1% 的约束违反（$\text{penalty} = 1{,}000 \times 0.01^2 = 0.1$）也无法对正向得分形成有效约束。上界 80,000 的设定考虑了搜索引擎的探索能力：当惩罚权重过高时（如 $80{,}000 \times 0.01^2 = 8.0$），轻微违反即产生远超正向得分的惩罚，搜索引擎被迫在极度保守的区域内搜索，可能错过具有轻微约束触碰但整体更优的参数配置。

4.8 B 国部署：从冷启动到推全

Exp-437160 是本文中唯一经历了从冷启动搜参到生产推全完整部署生命周期的实验。本节呈现完整的证据链。

4.8.1 冷启动信号

为验证 Sortify 是否能泛化到单一市场之外，我们在 B 国商品详情页以冷启动方式（无继承 Memory DB）部署了系统。首轮搜索结果如下：

表 8 | Exp-437160 冷启动结果（B 国，V1 第 1 轮）。

指标	值	约束/目标	状态
离线目标值	0.493	最大化	已达成
I_{gmv} 提升	**+4.9%**	$\geq +4\%$	满足
$I_{\text{ecpm_term}}$ 变化	-4.5%	$\geq -5\%$	临近边界
`ads_top10_rate`	—	受约束	满足
`ps_org_wc` 变化	**+271.2%**（1.0 → 3.71）	—	最大调整
`ratio_ads` 变化	-32.0%	—	广告份额被压缩

系统正确识别出 B 国基线排序公式中有机点击信号被严重**低估**（`ps_org_wc` 从 1.0 跳至 3.71，+271% 调整——所有实验中最大的单参数变化）。这一市场特异性洞察从首轮搜索中即涌现，无需任何先验校准，证明 Influence Share 框架能够泛化到新市场。然而，这一首轮方向性信号远非可部署的策略——后续搜参阶段才能找到经得起长周期验证的参数配置。

4.8.2 搜参历程：从方向到可部署参数

B 国实验经历了两个搜参阶段，最终找到可部署的配置：

V1（11 轮，03-05 至 03-07）。 初始冷启动搜参阶段的 GMV 胜率仅为 30%（均值 GMV 提升 −2.4%）。虽然系统成功识别了方向——有机点击信号被低估——但参数未能收敛。所有 7 个参数在 11 轮中始终在搜索空间内大幅振荡。V1 的核心教训是：*方向正确不等于参数可部署*。

V3（12 轮，03-10 至 03-13）。 在基于 V1 观察缩窄搜索空间并调整约束配置后，系统重启进入第二搜参阶段。12 轮中大部分仍表现出广告-有机跷跷板效应——Ads Impressions 与 Organic Clicks 严格反向交替——这一结构性模式与 A 国实验中观察到的钟摆效应（Section 4.5.1）一致。

R7 参数冻结（03-14）。 在 V3 的 12 轮中，R8 观测窗口（测量 R7 参数）产生了唯一一轮 GMV 和 Ads Revenue 同时为正的结果：GMV +2.6%、Ads Revenue +6.1%。基于这一帕累托最优结果，R7 参数被冻结以进行扩展验证。

4.8.3 胜出参数结构

冻结的 R7 参数揭示了与 A 国实验在结构上截然不同的解：

表 9 | Exp-437160 冻结参数（R7，B 国）。

参数	值	解读
`ps_org_wc`	7.667	Organic Click 权重（主导）
`ps_ads_wc`	4.381	Ads Click 权重（高）
`ps_org_wg`	0.568	Organic GMV 权重（被压制）
`ps_org_wo`	0.805	Organic Order 权重（被压制）
`ps_ads_wg`	0.641	Ads GMV 权重（被压制）
`ps_ads_wo`	0.957	Ads Order 权重（中等）
`ps_biz_price_pow`	1.047	价格幂次（近中性）

这组参数呈现出“高 Click、低 GMV/Order”的结构特征——排序公式主要由点击信号驱动，成交类信号被刻意压低。这种结构减少了自然排序与广告排序之间的跷跷板效应：当排序公式不过度放大 GMV/Order 信号时，自然商品不会因为“更能卖”而大量挤占广告位，从而使 GMV 和 Ads Revenue 能够同时正向提升。

这与 A 国热启动实验中观察到的残余钟摆效应（Section 4.5.1）形成对比：A 国的关键 trade-off 仍然集中在广告订单权重上，而 B 国 R7 参数则通过将主要排序信号转向 Click 来绕过这一冲突——Click 信号在自然渠道和广告渠道之间的零和性远低于成交信号。

4.8.4 长周期 A/B 验证

冻结的 R7 参数通过 7 天 A/B 测试进行验证（03-15 至 03-22；推全 Snapshot #38047，10% 实验组 vs 20% 对照组）。

表 10 | 7 天 A/B 核心指标（YMAL 场景，yin_exchange vs Control）。

指标	差异	备注
GMV/UU	**+4.15%**	核心 KPI
GMV	**+4.10%**	与 GMV/UU 一致
GMV/Order	**+3.97%**	客单价驱动 GMV 增长
Order/UU	+0.17%	近乎中性
Ads Revenue	**+3.58%**	广告收入同正
CPC	**+4.19%**	单次点击价值提升
Take Rate	**+3.01%**	变现效率改善
Click/UU	+1.21%	用户参与度提升
Ads Load	−2.64%	广告位减少但价值更高

GMV 增长主要由客单价驱动（GMV/Order +3.97%）而非订单量（Order/UU +0.17%），表明以 Click 驱动的排序将更高价值的商品推至更高位置。Ads Revenue 在 Ads Load 下降 2.64% 的情况下仍正向增长 +3.58%，说明单次广告价值显著提升（CPC +4.19%）——更少但更有价值的广告展示。7 天的观测窗口足以过滤日内波动，提供了比单轮观测更强的部署信心。

4.9 效率与运营成本

要点：Sortify 以极低的资源开销运行——每轮仅一次 LLM API 调用，无需 GPU，存储增长可忽略。主要成本是 3.5 小时的数据积累窗口，这由统计需求而非系统限制决定。系统在 Exp-401838 中连续运行 7 轮（25 小时），在 B 国两个搜参阶段累计执行 23 轮；这些实验中的停止与切换均源于治理层决策，而非系统故障。

需要区分自动化的边界。**内层循环**——参数搜索、在线部署、A/B 观测、LLM 校准和下一轮启动——无需人工干预即可运行。但**外层治理循环**——冻结哪一轮的参数进行扩展验证、是否根据长周期 A/B 结果推全至生产环境、以及推全后的监控——仍由人工驱动。B 国部署清晰地展示了这一边界：内层循环在两个搜参阶段自主执行了 23 轮，但冻结 R7 参数、执行 7 天静态 A/B 测试、以及批准生产推全的决策均由人工操作员基于 Snapshot 级 A/B 分析做出。这种分工体现了一个刻意的设计选择：系统自动化高频、数据密集的优化决策——人工延迟是瓶颈的环节——同时将低频、高风险的部署决策保留为人工治理。

与人工优化基线相比——通常每轮需要 1–2 人时用于数据分析、参数选择和部署验证——

表 11 | 运营效率指标。

指标	值	备注
总周期时间	**~4 小时**	3.5h 数据积累 + 25–50min 处理
日优化轮数	**~6 轮/天**	按 4h/轮节奏
每轮试验数	**5,000**	经 25 个并行工作线程，15–30min 搜索
人工干预（冷启动后）	**零**	内层优化循环内零干预
连续无人值守轮数	**7**（401838），**23**（437160，两阶段累计）	受限于实验时长与阶段切换，非系统故障。推全决策、参数冻结和长周期监控仍由人工管控
A/B 获取重试成功率	5 次 × 5 分钟间隔	生产中未观察到数据丢失
Memory DB 增长	~1 KB/轮	7 表，SQLite，存储开销可忽略
LLM API 调用	1 次/轮	单次元控制器推理

Sortify 将每轮内层循环的边际成本降至初始设置后约**零人时**，同时将优化节奏从约 1 轮/天（受限于人工可用性）提升至约 6 轮/天。这一提速不仅意味着更多的参数探索轮次，更关键的是使系统能够在日内流量模式变化时做出响应——例如早高峰、午间低谷和晚间高峰的流量结构差异可能影响最优参数配置，6 轮/天的节奏使每个主要流量时段至少能获得一次校准更新。

成本结构分析。 Sortify 的运行成本由三部分构成：(1) *计算资源*——Optuna 搜索在单台多核机器上运行，25 个并行工作线程消耗约 6.25–12.5 CPU 小时/轮（25 核 × 15–30 分钟）；(2) *LLM 调用*——每轮 1 次 API 调用，输入 4K–8K tokens，返回结构化 JSON。以 2026-03-26 OpenAI 官方公开价格计（GPT-5.4：2.50 美元/1M 输入、15.00 美元/1M 输出；GPT-5.3-Codex：1.75 美元/1M 输入、14.00 美元/1M 输出），在 `high` 推理模式下，计费输出包含 reasoning tokens，单轮成本约 0.03–0.10 美元，按每天 6 轮计合 **0.18–0.60 美元/天**。若通过 ChatGPT 账号登录使用 Codex CLI，成本以套餐额度消耗而非逐次 API 结算；(3) *数据积累窗口*——3.5 小时在线观测为统计需求决定的固定成本，非系统瓶颈。存储开销可忽略：Memory DB 每轮新增约 1 KB，30 轮后总大小仍低于 100 KB。

5. 讨论：一人之力的可行性与意义

Sortify 有一个值得专门讨论的维度——开发过程本身：整套系统的算法设计、代码实现、实验运行、数据分析、报告撰写和视觉素材制作，均由一个人调度一组 AI 编程智能体完成。没有一行生产代码出自人类之手；本报告亦没有一个字由人类起草。人类的角色自始至终是*架构师与编排者*：定义目标、分解问题、审查产出、引导智能体趋向一致的执行。本节审视这一单人模式的可行性、运作机制及其更广泛的意义。

5.1 交付物的完整范围

为使上述论断具象化，有必要逐项列举 Sortify 项目的全部交付物：

- **算法设计**。双通道 Belief/Preference 解耦、基于 LMS 的迁移校准、LLM 元控制器的结构化提示协议、非对称惩罚更新规则（Section 2）。
- **生产代码库**。 10 步单次循环 Pipeline、YOLO 持续运行循环及故障恢复、7 表 Memory DB Schema、Optuna 搜索引擎集成、Redis 参数发布接口（Section 3）。
- **实验运行**。跨两个市场的 30 轮优化（A 国 7 轮、B 国 23 轮），包括持续 25 小时以上的无人值守 YOLO 运行与完整的跨市场部署链路（Section 4）。
- **分析与报告**。在线 A/B 结果的量化评估、参数稳定性分析、LLM 校准收敛追踪、跨市场泛化评估，以及完整的中英双语技术报告。
- **展示素材**。程序化视频合成、图表生成及用于成果传播的视觉资产。

在传统团队模式下，这一范围通常需要 3~5 个角色分担：算法研究员、软件工程师、运维专员、数据分析师和技术写作者。将上述角色压缩为单人的关键，不在于个人产出效率的超常发挥，而在于工作基本单元从*编写*到*指挥*的根本转变。

5.2 Sortify 库的技术深度与开发复杂性

上述交付物清单仅从宏观视角概述了项目范围，但并未揭示 Sortify 作为一个独立软件库所承载的技术深度。事实上，Sortify 远非一组脚本的松散集合，而是一个包含近十万行代码、78 个 Python 模块、7 大子系统的生产级框架——其功能覆盖从底层数据加载到高层自主决策的完整技术栈。

参数搜索引擎。 Sortify 的核心功能之一是一套高度可配置的参数搜索（搜参）引擎。该引擎构建了依赖感知的公式计算图：给定一组排序公式及其嵌套的权重变量，系统自动解析依赖关系、构建分层计算图，并在权重变动时仅增量更新受影响的变量——避免了约 99% 的冗余计算。在此之上，系统集成了 Optuna 多目标优化框架，支持 NSGA-II 的 Pareto 前沿搜索、贝叶斯优化、进化策略和遗传算法等多种搜索策略。搜参过程还内置了多种相关性分析器（Pearson、Spearman、Kendall）、指标融合方法（加权均值、L2 范数、RMS 等 6 种），以及基于 Top-K 的排序质量评估——形成了一套从数据加载、公式计算、参数优化到结果分析的端到端闭环。

影响力置换算法。 Sortify 中最具原创性的技术贡献之一是**影响力置换**（Pairwise Influence）指标体系。该算法解决了一个关键问题：在多因子排序公式中，各权重因子对最终排序结果的实际影响力如何量化？传统的敏感度分析仅衡量参数扰动对单一指标的边际效应，而影响力置换算法将分析粒度下沉到*逐对比较*层面——从 Top-K 列表中按可配置策略（相邻对、随机对、全排列）采样物品对，计算每对物品在各因子维度上的差异向量 Δx_{ij}，再通过 $\text{Share}_f = |w_f \cdot \Delta x_{ij,f}| / \sum_k |w_k \cdot \Delta x_{ij,k}|$ 得到每个因子在该对比较中的相对贡献份额，最终以加权聚合（支持指数衰减、逆序衰减等加权策略）得到全局影响力分数 $I_f \in [0, 1]$。更进一步，算法还计算因子间的*冲突率* $\text{CR}_{a,b}$（两个因子对同一对物品产生相反排序倾向的频率），为权重调优提供了不可替代的诊断信息。这一算法从概念提出到数学形式化、再到向量化实现和生产集成，全程由 AI 智能体在人类指导下完成。

全自动运行管线。 在搜参引擎之上，Sortify 还构建了一套完整的自主运行管线。其核心包括两个层次：*OneShotPipeline*——将离线优化、LLM 提案生成和在线监控编排为一次性自动流程；以及 *YoloRunner*——一个"持续自主运行"的智能体循环，具备 Episode 状态追踪、离线到在线的工作流衔接、自动回滚决策和在线 Belief/Preference 状态管理。为支撑这一自主运行，系统实现了一个基于 SQLite 的 7 表持久化记忆系统（支持 DualMemory 双后端架构），记录每一轮优化的离线结果、部署配置、在线指标和决策依据，为 LLM 元控制器的提案生成提供完整的历史上下文。这意味着 Sortify 不仅是一个被调用的工具库，更是一个能够自主发起、执行、评估和迭代优化循环的智能系统。

开发复杂性。 上述功能的实现难度不容低估。仅就技术栈的多样性而言，Sortify 横跨数据工程（Parquet 增量加载与流式处理）、数值计算（NumPy 向量化的公式求值与增量更新）、优化理论（多目标 Pareto 优化与约束搜索）、系统工程（SQLite 持久化、Redis 参数发布、Grafana 实时监控集成）、以及 AI 应用（LLM 提示工程、自主决策、记忆检索）五大领域。这些子系统并非独立运行，而是通过依赖图、事件流和状态机紧密耦合——任何一个模块的设计缺陷都可能在运行时引发级联故障。整个代码库约 98,000 行、50 余个核心类、10 余种分析器，这一规模的系统由单人指挥 AI 智能体从零构建并部署到生产环境，本身便是"一人之力"模式可行性的最有力证明。

5.3 从 ParaDance 到 Sortify：技术谱系与领域积淀

上一节展示了 Sortify 近十万行代码、78 个模块的工程复杂度。单人模式何以驾驭如此规模的系统设计？关键一环在于：这并非一次从零开始的冒险。Sortify 的设计理念、评估架构和优化方法论直接源自 **ParaDance**[2]——一个由同一作者创建的开源多目标参数优化库。理解这一技术谱系，对于把握 Sortify 背后的领域深度以及从通用调参工具到自主智能体驱动优化系统的演进轨迹，都不可或缺。

[2] https://github.com/yinsn/ParaDance

历时 20 个月的迭代历程。 ParaDance 于 2023 年 5 月启动，持续开发至 2025 年 1 月——历时 20 个月，累计 357 次提交、发布 40 余个版本（v0.1.1 至 v0.6.7），经历 6 个大版本纪元。项目从一个简单的 Lorenz 曲线采样工具，演化为一个包含 3,688 行源代码、48 个 Python 模块、6 个核心子系统的生产级框架：评估体系（16 个独立评估器）、优化引擎（基于 Optuna 的贝叶斯搜索与多目标聚合）、数据加载、流水线编排、采样和可视化。每一个大版本都标志着一次质变：v0.2.x 引入 Portfolio 可视化；v0.3.x 建立贝叶斯优化引擎；v0.4.x 构建了评估生态，包括基于归并排序的加权逆序对计算和 JSON 公式系统；v0.5.x 加入分阶段优化、多线程实时监控和侧信息重排序；v0.6.x 交付了断点续训和生产加固功能。

超越时代的设计选择。 ParaDance 中的若干架构决策，在 2023 年的 Python 数据科学工具中属于非主流做法，而后来逐步被更广泛的行业趋势所验证：

- **可插拔的评估器架构。** `BaseCalculator` 使用 `partialmethod` 将 16 个评估器以单继承链的方式挂载为实例方法，规避了 Mixin 方案带来的 MRO 复杂性。新增评估器仅需三步：编写独立函数、注册为 `partialmethod`、添加 flag 字符串——无需修改任何已有代码。
- **声明式 JSON 公式引擎。** 一套支持中间结果级联引用（`step1#intermediate` → `step2`）、条件分支（`if(cond, true_val, false_val)`）和沙箱化求值的公式链系统，使算法工程师仅通过修改配置即可迭代打分公式，无需发版部署。
- **量级感知的自适应权重搜索。** 基于 `log10` 量级自动对齐不同特征的搜索边界（如 CTR ∼0.01 与 GMV ∼10,000），免去了人工归一化的繁琐步骤。
- **带预热的分阶段优化。** 前 N 轮使用 warmup 公式引导搜索方向，过渡时锚定最优值确保搜索连续性——有效解决了冷启动阶段的搜索低效问题。
- **生产级韧性设计。** 基于 SQLite 的优化过程持久化、针对长跑任务的断点续训、Joblib 并行优化，以及在概率单纯形上进行 Dirichlet 约束权重采样。

这些设计选择——配置驱动的流水线、可插拔的评估体系、量级感知的搜索、持久化的优化状态——直接塑造了 Sortify 的架构。Sortify 中的 7 表 Memory DB 是 ParaDance 的 SQLite 持久化机制的概念后裔；双通道 Belief/Preference 解耦将 ParaDance 的多目标聚合扩展到了在线校准领域；而 LLM 元控制器则占据了 ParaDance 中分阶段优化和 warmup 公式曾在更原始层级扮演的角色。

行业采纳。 截至本文撰写时，ParaDance 在 PyPI 上的累计下载量已超过 101,000 次，被各大互联网公司的算法工程师广泛使用，成为生产环境推荐系统中事实上最广泛采用的调参工具之一。这既验证了其问题定义的准确性，也证明了 20 个月开发历程中所做架构决策的合理性。这一用户基础还为作者提供了来自不同生产环境的持续反馈——这些反馈直接影响了 Sortify 的设计优先级，尤其是对迁移校准、约束敏感度管理和最小化人工干预的自主运行模式的重视。换言之，ParaDance 为 Sortify 提供的不仅是可复用的代码资产，更是经过 20 个月生产打磨的领域判断力——正是这种判断力，使得人类编排者能够在智能体驱动的开发中做出正确的架构抉择，而非在试错中消耗有限的迭代预算。

5.4 编排模式

在项目各阶段中，一种一致的操作模式自然涌现。人类操作者的工作循环为：

1. **定义意图**——明确需要构建、修复或分析的目标，以自然语言表述并附带验收标准。
2. **分解为智能体任务**——将目标拆解为离散的、可独立执行的单元，适合并行分发给多个智能体。
3. **审查与集成**——检视智能体产出的正确性、一致性和与整体设计的对齐度，将通过审查的制品合并入项目。
4. **失败迭代**——当智能体产出偏离预期时，诊断失败原因是任务规约不清（人类失误）还是执行偏差（智能体失误），并相应调整。

这一循环与 Sortify 系统内部的 LLM 元控制器模式形成镜像：一个更高层级的智能（人类）不执行底层优化（写代码、起草文本），而是调整*框架级参数*（任务规约、设计约束）以引导智能体的执行方向。这种平行关系并非巧合——使 Sortify 自主优化得以运行的*掌舵而非亲为*原则，同样使单人开发模式成为可能。

5.5 元嵌套：智能体构建智能体

Sortify 项目展现出一种值得显式讨论的**元结构**嵌套闭环。该项目的独特之处不仅在于它*是什么*，更在于它*如何被创造*——两个层面形成了自指的递归结构。

第一层：目标域智能体。 在业务逻辑层面，Sortify 本身即是一个自主智能体——它被设计用来替代传统算法工程师在排序优化中的系统性决策工作。过去，算法工程师需要持续监控离线指标与在线大盘的偏差，手动调节惩罚项与融合公式，试图弥合认知偏差与价值失调。而 Sortify 基于 SEU 理论引入 Belief/Preference 双通道，自行观察 20 轮历史证据，自行诊断结构性断点，自行完成影响力置换——这是机器在特定领域（推荐系统调参）对人类脑力劳动的结构性替代。

第二层：创造域智能体群。 在开发层面，整套系统由单人调度一组 AI 编程智能体从零构建。这些智能体群体承担了传统团队中架构师、全栈工程师、数据工程师、QA 测试工程师乃至学术论文执笔者与图表绘制者的全部角色。它们共同完成了：搭建包含依赖感知计算图的 Optuna 搜参框架与端到端闭环流水线；设计并实现包含 7 张关系表的持久化记忆数据库；遵循学术规范将实验数据提炼为完整的中英双语技术报告。

递归闭环。 两层逻辑叠加形成了一个递归结构：*一组 AI 智能体（创造域）构建了另一个 AI 智能体（目标域，即 Sortify），而后者又自主地自动化了极其复杂的业务逻辑*。在这一过程中，人类的角色从“编写代码的工匠”跃迁为“制定规则、提供高维直觉、下发验收标准的总编排者”。这种“智能体构建智能体”的自举过程，不仅是一个有趣的结构同构，更是多智能体协作在处理兼具深层算法逻辑与学术严谨性要求的系统工程时可行性的直接证明。

5.6 意义与影响

范式重构而非效率提升。 当整个项目的算法设计、代码编写、实验落地、以至最终的中英双语 LaTeX 报告撰写与配图，全部由单人调度智能体完成且“零人类代码、零人类文字”时，这已完全超越了 Copilot 时代“代码补全”或“提效工具”的范畴。它标志着一种范式的重构：在传统软件工程中，从“有一个好点子”到“系统上线”之间横跨着巨大的实现鸿沟——即便提出了“基于 SEU 解耦信念与偏好”这样的核心直觉，仍需数月的基础设施搭建、数据清洗和 Bug 排查才能验证其价值。如今，实现成本趋近于零。决定项目上限的不再是开发者对框架 API 的熟练度或团队的人力编制，而是核心决策者对业务痛点的*理解深度*、对复杂系统的*架构品味*，以及对智能体协作的*编排艺术*。

复杂系统构建的壁垒消融。 构建 Sortify 这类生产级系统的传统壁垒，首要瓶颈并非智力而是*组织运作*：多人团队的协调成本、将设计意图转译为实现代码的沟通开销、以及从提出变更到看到效果的迭代延迟。当 AI 智能体吸收了实现层面的负担，约束瓶颈便从团队规模转移到单个人的*问题分解能力和设计判断力*。这意味着，深谙问题领域但缺乏完整工程团队的领域专家，如今能够构建和部署同等复杂度的系统——这是单个实践者可达边界的质变式扩展。

认知杠杆而非认知替代。 一个关键的细微之处在于：人类的贡献并非被消除，而是被*浓缩*。设计决策——为何选择双通道而非单通道、为何采用框架级控制而非直接参数操作、为何构建持久记忆而非无状态轮次——需要领域理解、架构品味和判断力，这些是智能体无法自主生成的。智能体消除的是将这些决策*转译*为可运行代码、经测试的 Pipeline 和格式化文档的机械劳动。因此，人类认知贡献相对于项目总产出的比率，在每单位人力投入上极高，但人力投入本身对项目成功不可或缺。这是*认知杠杆*而非认知替代。

可复现性与可审计性。 本项目中每一件制品都携带完整的来源链路：生成各代码模块的智能体对话、接受或拒绝中间产出的审查决策、以及迭代精炼至最终结果的完整历史。这种可追溯程度——吊诡地——比大多数多人团队工作流所能提供的*更加全面*，后者的设计思路往往散落在 Slack 线程、白板讨论和未记录的代码评审之间。单人编排模式将所有决策路由通过单一责任点，辅以 AI 辅助的记录保持，可能产出比传统流程更具可审计性的开发历史。

模式的可推广性。 此处展示的单人可行性并非孤立的奇观，而是一个随 AI 编程智能体成熟必将日趋常态化的模式的早期实例。使其成为可能的特定组合——界定清晰的问题域、模块化的系统架构、以及能够从自然语言规约生成正确代码的智能体——并非 Sortify 独有。任何可分解为范围明确、可独立测试的组件的项目，均适用于这一开发模式。限制因素不在于工具，而在于人类在心智中持有完整系统设计并以足够精度规约任务的能力。

本报告——包括此刻你正在阅读的这段话——由 AI 智能体在人类指导下撰写，这本身便是它所描述之论题的一个证明。人类创造力与机器执行力之间的边界并未消失，但已然位移：人类提供*做什么*和*为什么*；智能体提供*怎么做*。对 Sortify 而言，这一分工足以交付一套生产系统、跨两个市场部署上线，并产出本报告所呈现的全面评估与文档。未来的技术探索者，将不再被困于实

现的泥沼，而是成为纯粹的思想者与指挥官——这不是对效率的优化，而是对创造方式本身的重新定义。

6. 结论、局限与未来方向

本文提出了 Sortify——一个智能体驱动的闭环排序优化系统，将排序优化重构为持续的影响力置换过程，由 LLM 智能体掌舵，在各排序因子间动态调控影响力分配。Sortify 通过统一的闭环架构解决了传统人工优化的三个结构性缺陷：离线-在线迁移鸿沟、诊断信号纠缠和无状态逐轮操作。双通道自适应框架将迁移映射修正（Belief）与约束惩罚调整（Preference）解耦为正交维度；LLM 元控制器在框架级参数上对 LMS 校准残差进行证据驱动的补充修正；持久化 Memory DB 跨轮积累校准智能，支持热启动与跨市场迁移。系统已部署于 A 国和 B 国的大规模推荐平台，内层优化循环以 **4 小时**为周期自动运行，产出两个层次的证据：A 国热启动实验中的单轮峰值表现（GMV **+9.2%**、Orders **+12.5%**，且 LLM 修正项数在 7 轮内由 5 项收敛至 2 项），以及 B 国经部署验证的表现（7 天 A/B 验证 GMV/UU **+4.15%**、Ads Revenue **+3.58%**，已推全至生产环境）。

尽管取得了上述成果，Sortify 仍存在若干具体局限：

- **参数振荡未完全收敛**。在保留于正文的 A 国热启动实验中，`ps_ads_wo` 仍表现出 4.5 倍的残余钟摆效应（Section 4.5）。广告-有机 trade-off 所产生的多峰目标景观，是单目标加惩罚的建模方式只能近似而无法消解的根本原因。这种振荡说明系统虽然已从无界探索转向受约束摆动，但尚未收敛到唯一稳定操作点。
- **低流量窗口的统计不确定性**。最强结果（Exp-401838 R7 的 GMV +9.2%、Orders +12.5%）采集于约 12K impressions 的低流量凌晨时段（Section 4.3）。虽然系统运行正常，但这些估计值的统计功效低于典型流量时段的轮次。
- **极端结构性断点的实证证据不足**。当前保留的 A 国实验主要展示了 LLM 修正项数的收敛，而未覆盖一个足够长、足够剧烈的热启动后生命周期来验证大幅结构性断点的处理效果（Section 4.6）。这意味着本文对 LLM 元控制器的证据更偏向“降低持续校准工作量”，而非“处理极端突变”。

上述局限指向两个未来工作方向。第一，以**多目标 Pareto 前沿**方法替代单目标加惩罚的建模，将广告-有机 trade-off 显式建模为前沿而非单一最优点，有望从根本上消除钟摆效应。第二，开发**置信度加权迁移模型**，根据观测流量体量调整校准激进程度——在低流量窗口更保守、高流量时段更积极——在不牺牲收敛速度的前提下降低统计风险。这些扩展自然构建于 Sortify 现有的双通道和 Memory DB 架构之上，所需修改集中于目标函数建模和上下文组装，而非根本性的架构变更。

最后，Sortify 项目本身——从底层的算法推演、近十万行代码的工程实现、30 轮生产环境的优化与部署链路，直至本报告的落笔成文——全程由单人指挥 AI 智能体群落独立完成，未曾掺杂哪怕一行人类代码或一句人类底稿。这一事实本身，便是一场静默却震耳欲聋的实验。如 Section 5 所述，它揭示了一种“智能体创生智能体”的元嵌套逻辑：处于创造域的 AI 实体孕育了目标域的工业级 AI 中枢（Sortify），而后者又继续向下接管了极度复杂的商业运转。这绝非传统意义上工作流的“降本增效”，而是一次对软件工程与人类探索边界的结构性重塑——当构想与现实之间漫长而泥泞的实施鸿沟被彻底填平，一个残酷而迷人的新纪元已然开启：未来决定创造力上限的，将不再是庞大团队的执行规模，而是孤独决策者的认知穿透力与万物皆可编排的艺术。

参考文献

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

James O Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2nd edition, 1985.

Miroslav Dudik, Dumitru Erhan, John Langford, and Lihong Li. Doubly robust policy evaluation and optimization. *Statistical Science*, 29(4):485–511, 2014.

Matthias Feurer, Jost Tobias Springenberg, and Frank Hutter. Initializing bayesian hyperparameter optimization via meta-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015. doi: 10.1609/aaai.v29i1.9354.

Alexandre Gilotte, Clément Calauzènes, Thomas Nedelec, Alexandre Abraham, and Simon Dollé. Offline a/b testing for recommender systems. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, pages 198–206, 2018. doi: 10.1145/3159652.3159687.

Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

Xiao Lin, Hongjie Chen, Changhua Pei, Fei Sun, Xuanji Xiao, Hanxiao Sun, Yongfeng Zhang, Wenwu Ou, and Peng Jiang. A pareto-efficient algorithm for multiple objective optimization in e-commerce recommendation. In *Proceedings of the 13th ACM Conference on Recommender Systems*, pages 20–28, 2019. doi: 10.1145/3298689.3346998.

Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3):225–331, 2009.

German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

Shubhra Kanti Karmaker Santu, Parikshit Sondhi, and ChengXiang Zhai. On application of learning to rank for e-commerce search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 475–484, 2017. doi: 10.1145/3077136.3080838.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhen-Hua Wei, and Hengshu Zhou. A

survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18 (6), 2024. doi: 10.1007/s11704-024-40231-1.

A. 贡献者

Yin Cheng（yin.cheng@shopee.com）及其 AI Agents 负责了本项目的全部核心工作：

- **项目开发**：系统架构设计与全栈工程实现
- **算法设计**：影响力置换排序算法（Pairwise Influence）的提出与迭代
- **实验实施**：A/B 实验的部署、监控与数据分析
- **报告撰写**：本技术报告的构思、写作与修订

致谢

特别鸣谢 Liao Zhou（liao.zhou@shopee.com）在项目推进过程中的深度参与——围绕实验方法论与开发环境演进，提供了持续的讨论反馈与经验传递，对方案的迭代优化起到了重要作用。

同时感谢以下同事在项目推进过程中给予的支持：

xiyu.liang@shopee.com　kailun.zheng@shopee.com
dihao.luo@shopee.com　tewei.lee@shopee.com
zhangweiwei@shopee.com　mark.cai@shopee.com
jian.dong@shopee.com　andy.zhanggx@shopee.com

B. 实现细节

B.1 A/B 实验集成

Sortify 集成于平台现有的 A/B 测试基础设施，而非部署独立的实验框架。每次参数推送至 Redis 后，平台自动在对照组（当前生产参数）和实验组（Sortify 优化后的参数）之间进行流量分配。A/B 报告由平台标准指标管道生成，由 Sortify 的单次循环 Pipeline 在步骤 1 中拉取。

时间窗口对齐。所有观测窗口对齐到 15 分钟边界，以匹配平台指标聚合的周期。最小 3.5 小时的积累窗口确保在大多数时段条件下有足够流量体量用于稳定的提升估计。最大窗口同样限定为 3.5 小时，以防止较早流量模式的陈旧数据污染后续观测。

B.2 Redis 参数发布

7 参数向量序列化为 JSON 并写入排序服务监控的 Redis 键。发布操作是原子性的——要么 7 个参数同时更新，要么全部不更新。回滚机制允许通过重新发布存储在 `yolo-state.env` 中的 `PREV_PARAMS_JSON` 回退至上一组参数。

B.3 状态文件管理

两个状态文件维护跨 Pipeline 执行和系统重启的连续性：

`yolo-state.env` —YOLO 循环状态：

```
YOLO_ROUND=5
LAST_PUSH_TIME=2026-03-05T18:45:00
LAST_ONLINE_FROM=2026-03-05T15:15:00
```

```
LAST_ONLINE_TO=2026-03-05T18:45:00
PREV_PARAMS_JSON={"ps_ads_wo": 3.51, ...}
PREV_AB_REPORT_PATH=/path/to/report
```

one-shot-latest.env —上次成功的单次循环交接：

```
LATEST_RUN_TAG=20260302_143022
CONFIG_PATH=.../pdp-th-topk-order-guard-agent.next.json
BEST_SO_FAR=.../search-artifacts/*_best_so_far.json
```

B.4 目录结构

每轮产出的工件存放在带时间戳的目录中：

```
updates/
|-- memory/
|   `-- agent_memory.db             # 主数据库（跨轮共享）
|-- <run_tag>/
|   |-- online-loop.log             # 10 步 Pipeline 输出
|   |-- search.log                  # Optuna 试验结果
|   |-- ab_report.md                # 在线 A/B 指标
|   |-- llm_proposal.json           # LLM 输出（含证据）
|   |-- rule.publish.json           # 已发布参数
|   |-- search-logs/
|   |   `-- *_trial_table.csv       # 全部 5,000 次试验结果
|   `-- search-artifacts/
|       `-- *_best_so_far.json      # 找到的最优参数
|-- yolo-state.env
|-- yolo-event-log.md               # 所有轮次的审计日志
`-- one-shot-latest.env
```

C. 符号表

表 12 | TAB-APP-02. 符号定义。

符号	定义	引入位置
$\pi_q(\boldsymbol{\theta})$	request q 在参数 $\boldsymbol{\theta}$ 下的排序置换	Eq. 1
S_{n_q}	作用在 n_q 个商品上的对称群	Eq. 1
$\mathbf{S}_q(\boldsymbol{\theta})$	request q 的总分向量	Eq. 2
H_{qij}	商品 i, j 的比较超平面 $x_i = x_j$	Eq. 2
C_π	与置换 π 对应的排序 chamber	Eq. 2
$\boldsymbol{\theta}$	7 维排序参数向量	Eq. 10
$\mathcal{F}$	全部排序因子集合	Eq. 4
$S_q(i;\boldsymbol{\theta})$	商品 i 在 request q 上的总分	Eq. 3
$\Delta_q(i,j;\boldsymbol{\theta})$	商品对 (i,j) 的总 pairwise margin	Eq. 3
$\Delta_{q,f}(i,j;\boldsymbol{\theta})$	因子 f 在商品对 (i,j) 上贡献的分数差值	Eq. 4
$Z_{qij}(\boldsymbol{\theta})$	商品对 (i,j) 的总影响预算	Eq. 6
$s_{q,f}(i,j;\boldsymbol{\theta})$	因子 f 在商品对 (i,j) 上的成对影响力份额	Eq. 7
P_q	request q 中参与聚合的 informative / Top-sensitive pair 集	Eq. 9
r_{qi}	商品 i 在 request q 中的 rank	Eq. 8
w_{qij}	商品对 (i,j) 的基于排名的指数衰减权重	Eq. 8
τ	商品对权重的衰减率参数	Eq. 8
$I_f(\boldsymbol{\theta})$	因子 f 的聚合 Influence Share	Eq. 9
$\mathcal{J}(\boldsymbol{\theta})$	复合搜索目标函数	Eq. 10
λ_j	约束 j 的惩罚权重	Eq. 10
$v_j(\boldsymbol{\theta})$	约束 j 的违规幅度	Eq. 10
α	迁移模型斜率（Belief 通道）	Eq. 12
β	迁移模型截距（Belief 通道）	Eq. 12
η	LMS 学习率（$= 0.2$）	Eq. 13
e_t	第 t 轮的迁移预测误差	Eq. 13
$\Delta\beta_{\mathrm{LLM}}$	LLM 截距调整量（$\in [-0.1, +0.1]$）	Eq. 15
p_j	约束 j 的归一化违规压力	Eq. 16
δ_{up}	惩罚收紧步长（$= 0.25$）	Eq. 17
δ_{down}	惩罚放松步长（$= 0.08$）	Eq. 17
m_{LLM}	LLM 惩罚乘数（$\in [0.5, 2.0]$）	Eq. 20

表 13 | 缩写表。

缩写	全称
GMV	交易总额（Gross Merchandise Value）
PDP	商品详情页（Product Detail Page）
TPE	树结构 Parzen 估计器（Tree-structured Parzen Estimator）
LMS	最小均方（Least Mean Squares）
YOLO	You Only Live Once，持续 Pipeline 模式
KPI	关键绩效指标（Key Performance Indicator）
CTR	点击率（Click-Through Rate）
eCPM	每千次展示有效成本（Effective Cost Per Mille）
ROI	投资回报率（Return on Investment）